

CASEPATH: AN AGENTIC, PROCESS-FIRST ARCHITECTURE FOR DETERMINING EVIDENCE REQUIREMENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

LLM agents increasingly act in rule-governed workflows, where an action is right only if the process state makes it applicable. In regulated work such as insurance claims, one fact can make a receipt required, irrelevant or premature. Direct generation is then risky: even with the rules in context, one output judges the case and chooses the action, and can mix branches without checkable reasons. CASEPATH, a process-first agentic architecture, splits the work: language models judge conditions, and executable control activates applicable requirements, derives their evidence needs and turns open conditions into questions; each request keeps its reason. When two insurance cases differ in one branch-deciding fact, CASEPATH makes about half as many unjustified checklist changes as direct generation, and about a third as many as a model given CASEPATH’s graph. On synthetic tenancy claims with judgments held fixed, enforcing the enclosing process more than halves the documents requested now and keeps every justified one. Coverage does not rise, as it depends on reading the rules and the case. We release both synthetic datasets, with structured references, for training and evaluating agents. Enforced process state gives agents a control layer between judgment and action.

1 INTRODUCTION

Language-model agents increasingly act inside rule-governed workflows such as customer service (Yao et al., 2025) or claims handling, where an action is right only if the current state of the process makes it applicable. The usual design asks the model for the action directly, sometimes with the rules or a process graph in its prompt. One output must then judge the case and decide what the process requires. It records neither decision, so it can mix requirements from different branches, and nobody can check which actions are justified. Where the process is binding, as in insurance claims, such mistakes are costly: a claimant is asked for documents that do not apply, or a document is dropped without a reason.

We isolate and study one such action: the request for evidence. Suppose a watch is stolen in a household burglary. Should the insurer ask for proof of its value? That depends on a fact the claim category does not reveal: was the watch separately insured above the policy threshold? If it was, its value must be established, and a receipt or an expert’s valuation can show it. If it was not, neither document is needed, and if nobody knows yet, the insurer should first ask. Each answer puts the case on a different branch, so the same receipt can be required, irrelevant or premature. A document on file can also miss the point: a photograph shows that the watch existed, not what it was worth.

A document matters only because an active requirement needs a fact that it can establish. Judging the facts is a question about the case, and deciding what the case requires is a question about the process, so CASEPATH gives the two to different components (Figure 1). Language models read the rules and judge the case. Executable process control decides which requirements are active and what evidence each needs, turns open conditions into questions and keeps each request’s reason.

We contribute in four ways:

1. **Architecture.** We introduce CASEPATH, a process-first control layer between language-model judgment and agent action that derives every request or question, with its reason, from the process state (Section 3, Algorithm 1). We are also building it into claims-handling software.

2. **Evaluation.** We report two tests that compare process control with common designs, such as direct generation and a process graph in the prompt (Section 4). In Study A (insurance), two cases differ in one fact that decides a branch, and only the documents that branch governs should change. In Study B (tenancy), with the model’s judgments held fixed, enforcing the enclosing process should remove requests that do not apply and keep the justified ones.
3. **Datasets.** We release the synthetic cases of both studies for training and evaluating agents: Study A’s 36 pairs of theft claims, each with the documents its one changed fact requires, and Study B’s 150 tenancy claims, each pairing a customer message and any attachments with a reference of process, obligations, document states, next action and sources (Figures 3 and 4).
4. **Empirical insight.** We show that process control enables selectivity. In Study A, CASEPATH cuts unjustified checklist changes by almost half against the Direct baseline and by almost two thirds against Graph context, which has CASEPATH’s graph in its prompt (Table 2). In Study B, inherited conditions more than halve the requests made now and keep every justified one (Figures 7 and 8). Coverage does not rise, as it rests on reading the rules and the case (Tables 2 and 5). Every request carries its chain, so each Study A error traces to its source, most often a rule that can be corrected once (Appendix B.2).

The benchmarks, saved outputs and evaluators behind every number are released.¹

2 RELATED WORK

Rule and case-management languages (OASIS LegalRuleML Technical Committee, 2021; Merigoux et al., 2021; Object Management Group, 2024; 2016; Vaculín et al., 2011; López et al., 2020) and proactive enforcement (Basin et al., 2024) encode or apply rules, but none reads a free-text case. PAGED and WorFBench test extracting or writing procedures (Du et al., 2024; Qiao et al., 2025), and neuro-symbolic systems have a model formalise a problem for an exact solver (Pan et al., 2023; Guan et al., 2023). In CASEPATH a model judges each case, and code applies the process.

Process-aware agents follow a workflow or policy in the prompt (Xiao et al., 2024; Yao et al., 2025), a hand-built state machine (Wu et al., 2024), guidance from a procedural graph (Lu et al., 2026) or a compiled procedure paged frame by frame (Yu et al., 2026), and guardrails check another agent’s actions (Chen et al., 2025; Xiang et al., 2025). In all of them a model picks the actions, though StateFlow fixes a few in advance; in CASEPATH code decides every request from the model’s judgments. The closest work, an evidence ledger (Kim, 2026), also holds a branch’s obligations until its condition resolves, but a model plans them per query, and it names no inherited conditions.

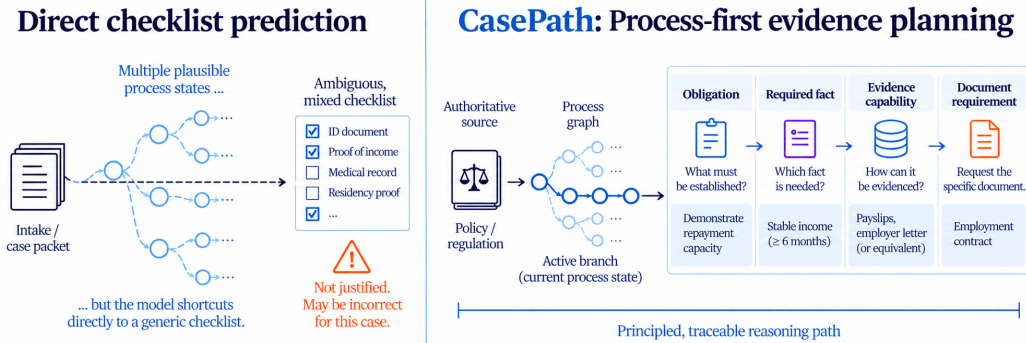


Figure 1: **CASEPATH derives each document from the active process.** On a loan application: direct prediction can mix branches; CASEPATH follows the active one from obligation to fact, evidence and document.

¹Code and data: <https://anonymous.4open.science/r/casepath-9673/>.

Algorithm 1 CASEPATH: the model judges the case; code decides every question and request.**Input:** the rules; a case file**Output:** questions, and document requests that each keep their reason

```

1: turn the rules into a process of branches and obligations ▷ once
2: mark each condition true, false or unresolved, quoting the case ▷ model
3: for each obligation do
4:   take its condition and those of all enclosing branches ▷ full process scope
5:   if any is false then skip it ▷ irrelevant
6:   else if any is unresolved then ask what would settle it ▷ premature
7:   else for each fact, request what its closest route still lacks ▷ required

```

Why these agents are not our baselines. None takes a process and a case and returns document requests: most return actions, some need inputs our cases lack, such as a hand-built state machine or another agent’s actions, and some have no public code (Appendix E). Our baselines test their shared ideas: Direct reads the sources as a τ -bench agent reads its policy, Graph context adds CASEPATH’s graph as FlowBench adds a workflow, every Study B baseline reads the compiled process, Rule-first has the model write and run its own rules, and Evidence-first and Document-first list evidence needs first, like the ledger’s planner. We did not rebuild their runtimes or claim to beat them.

Information-seeking agents retrieve (Asai et al., 2024) or ask (Li et al., 2024; Zhang et al., 2024; Taranukhin et al., 2026) to complete an answer or a plan, and legal work benchmarks evidence reasoning in litigation (Bi et al., 2026) or assesses compliance (Chung et al., 2025). CASEPATH ties each request to an active obligation and each question to an open condition.

3 SYSTEM ARCHITECTURE

CASEPATH splits the work in two: language-model agents judge what is true of a case, and executable process control derives what the case requires, with the process state kept explicit between them.

The rules become a process of nested branches, each guarded by a condition and holding requirements, or *obligations*. An obligation needs a *required fact*, such as the watch’s value. An *evidence capability*, here proof of value, can establish that fact. Each allowed set of documents that supplies it, such as the receipt or an expert’s valuation, is a *route*, and one route is enough. Where the rules are not yet formal, agents read the source texts into this structure. For each case they mark every condition true, false or unresolved, quoting the case, and judge the documents on file.

Code does the rest (Algorithm 1; formal version in Appendix A). An obligation applies only if its own condition holds and so does that of every branch enclosing it, which we call *full process scope*. Proof of the watch’s value is thus required, irrelevant or premature when its condition is true, false or unresolved. Checking the enclosing branches can remove or defer a request that the obligation’s own condition allows, and can never add one. A timing condition can also hold a request back. For each fact, code picks the route the file comes closest to completing, its *closest route*, and requests only what is missing, so each request keeps its chain from source to document.

In Figure 2, a tenant reports mould in the bedroom and attaches the email that notified the landlord. Suppose a landlord notified of a defect must assess and address it. The email proves the notice but not how far the mould has spread, the one fact still open. Photographs or an inspection report can show it, so the tenant is asked for one.

Software. We are building CASEPATH into agentic claims-handling software, under active development.² The released version is a local workbench for handling Study B’s 150 claims; it also hosts Study A’s method, which reproduces all 72 recorded cases from their saved judgments. The results below measure the method, not the software (Appendix Table 7).

²Software: <https://anonymous.4open.science/r/casepath-9673/casepath/>.

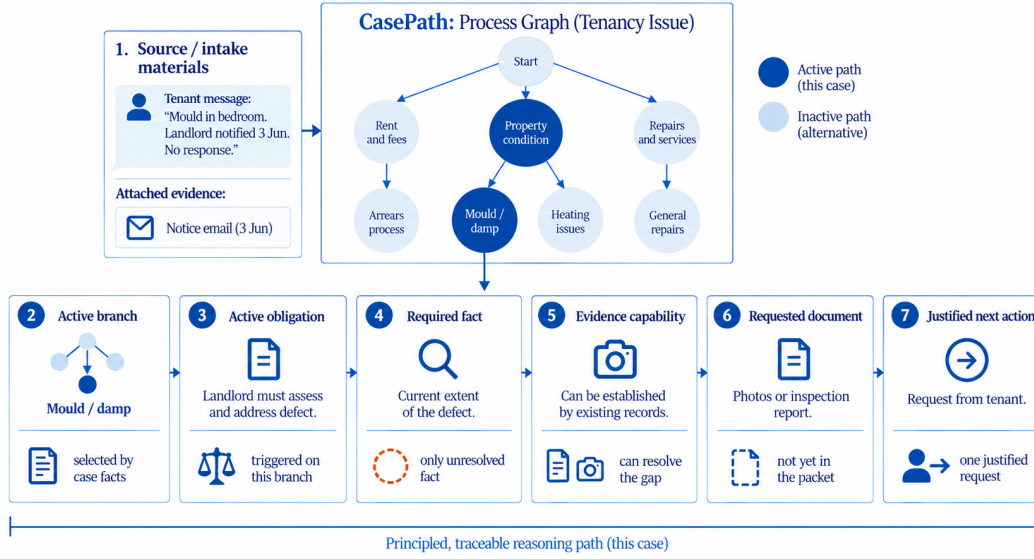


Figure 2: **The one open fact decides what to request.** An illustrative process: the case facts select the mould branch (dark nodes), and the rule is hypothetical.

4 METHODOLOGY

Process control makes two predictions, and each study tests one (Table 1). **RQ1:** when a branch-deciding fact changes, do the requests change where that branch requires and nowhere else? **RQ2:** with the model’s judgments held fixed, do conditions inherited from enclosing branches remove requests that do not apply and keep justified ones? The studies share no data or code (Appendix A). We release both datasets under CC BY 4.0, with datasheets and verification scripts (Figures 3 and 4; Appendix D). Their cases are synthetic; the law, policy terms and forms they use are real.

4.1 STUDY A: ONE CHANGED FACT

Agents built the rules from federal law (Swiss Confederation, 2024), four insurers’ policy terms (CSS Versicherung AG, 2024; AXA Versicherungen AG, 2021; Generali Allgemeine Versicherungen AG, 2026; Simpego Versicherungen AG, 2024), a claim form (CSS Versicherung AG, 2026) and industry model conditions (Schweizerischer Versicherungsverband SVV, 2022), turning 308 statements quoted from 197 passages into conditional rules over a fixed catalogue of documents. Study A runs a variant of Algorithm 1: each rule checks only its own condition, and every route of every active rule is requested (Appendix A). We took 9 branch conditions from the sources, such as whether a stolen item was separately insured, and set each in four neutral household-theft stories. Each pair’s reference, derived by code from an agent-built contract of the sources that no legal expert reviewed, lists the justified document changes. Figure 3 shows the 36 released pairs and opens one: a short theft story

Table 1: **Each study tests one prediction of process control.**

	Study A	Study B
Question	RQ1: does one changed fact cause the right checklist change?	RQ2: does the enclosing process remove requests that do not apply?
Cases	Household-theft insurance claims	Tenancy disputes
Rules come from	Agents reading law and policy texts	Code compiling synthetic templates
What is scored	Changes between the checklists of two cases	Complete lists of requests
Form of Algorithm 1	Own conditions only, every route	Inherited conditions, one route

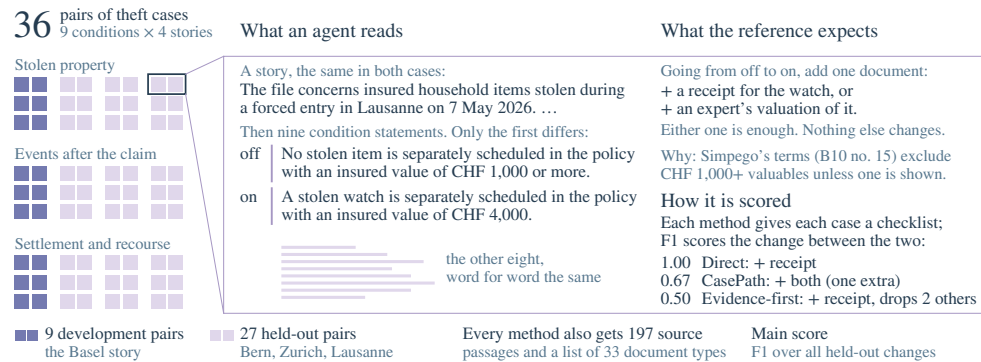


Figure 3: **A Study A pair tells one theft claim twice with one condition changed, and its reference lists only the documents that this change justifies.** Rows are the nine conditions and columns the four stories, with two squares per pair, one per case (dark: development); the outlined pair (scheduled valuable, Lausanne story) is opened at right.

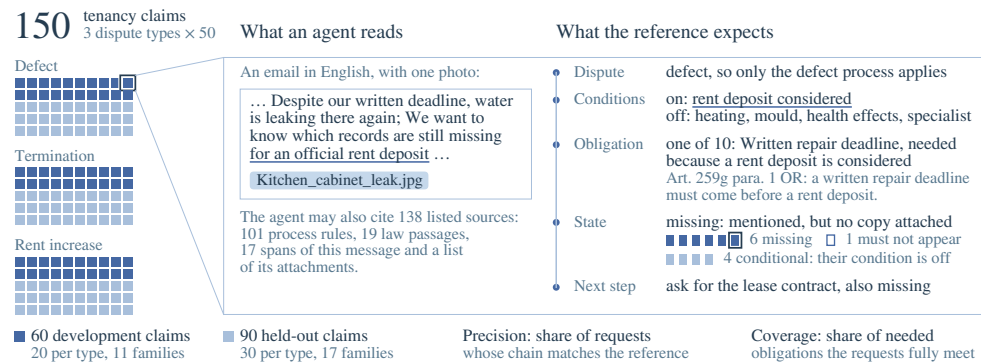


Figure 4: **A Study B claim is one tenant's message, and its reference runs from the dispute type to the next step.** Each square is one claim (dark: development). A family groups near-duplicate claims, and no family crosses the split. Claims arrive by email, portal, chat or phone note; 75 are in English and 75 in Swiss Standard German. The outlined defect claim is opened at right and followed through one of its ten obligations, whose document is outlined in the state row; the next step applies to the whole claim. The underlined words in the message set the underlined condition.

followed by nine condition statements, told twice with one statement changed. When a stolen watch is separately insured for CHF 4,000, the reference adds a receipt or an expert's valuation (either is enough) and changes nothing else. Development used one story's 9 pairs; the other 27 were read once, after the method, data, analysis and criteria were frozen.

A *checklist change* is a document added or withdrawn between the two checklists of a pair. It is *correct* if the sources justify it and *wrong* if not, and a justified change not made is *missed*. Three baselines share the model (openai/gpt-5.6-terra, temperature zero), sources, catalogue and cases but write the checklists themselves: *Direct* from the case and the sources, *Graph context* with CASEPATH’s graph and rules in its prompt, and *Evidence-first* after listing the evidence it needs and retrieving passages. Appendix B gives the statistics, the eight predeclared criteria and the costs. A wrong document on both checklists never shows up as a change, so Study B scores complete lists.

4.2 STUDY B: SAME JUDGMENTS, WITH AND WITHOUT THE ENCLOSING PROCESS

In Study B, code compiles the process from three synthetic tenancy templates, agents only judge the case, and the planner applies full process scope (Algorithm 1). The one condition an obligation inherits is the claim’s dispute type (a defect, a termination or a rent increase), so a document that only

a defect dispute needs is requested only in a claim judged a defect. With every timing condition true and one route per capability, Study B does not test timing or route choice (Appendix A).

The benchmark has 150 synthetic claims from 28 families of near-duplicate claims; 60 served for development and 90 were held out. Figure 4 opens one. An agent reads the tenant’s message and any attachments; the reference records the dispute type, the branches that apply, the chain from each obligation to the document that meets it, the state of each document and the next step. In the opened claim the tenant mentions a written repair deadline without attaching it, and the reference marks that document missing. Five pipelines share the process, case file, model (`openai/gpt-5.6-terra`) and 2 calls per claim. CASEPATH assembles its output in code from the model’s judgments; four baselines have the model write it, directly (Direct) or after first drafting the documents (Document-first), the process branches (Process-context) or condition-and-action rules (Rule-first). Two controls reuse CASEPATH’s saved judgments: *Local scope* drops the inherited conditions, and *Compiled control* recomputes the same decisions in another form and matches exactly.

The benchmark’s own evaluator scored no output, as its reference scenarios leave the pipelines’ condition names unassigned, so none of the 12 predeclared targets was met (Appendix C). Without new model calls, we therefore compare CASEPATH with Local scope at the same saved judgments and score every pipeline’s requested documents against the reference (*document-request scoring*). A request is *valid* if its chain from obligation to fact to evidence matches the reference. *Justified-request precision* is the share of requests that are valid, and critical-evidence recall, or coverage, is the weighted share of required evidence that a complete requested route covers. The held-out claims were seen during development, so they are not a fresh test (Appendix C.5).

5 RESULTS

5.1 STUDY A: CASEPATH CHANGES FEWER DOCUMENTS WITHOUT A REASON

Against Direct, CASEPATH makes 12 fewer wrong changes (15 against 27) at the cost of one correct change (24 against 25), and it gets the most pairs exactly right (Table 2).

The baselines’ extra errors are withdrawals and documents that no branch governs. No pair requires a withdrawal, so the benchmark cannot credit a correct one; 11 of the 12 errors that CASEPATH avoids against Direct are withdrawals, and CASEPATH withdraws nothing (Figure 5). All 120 wrong changes of the baselines are on documents no branch governs, against 9 of CASEPATH’s 15 (Figure 6A). The baselines write each checklist afresh, so general documents drift. CASEPATH’s changes follow the condition: reassigned to other conditions, their F_1 falls from 0.667 to at most 0.083 (Figure 6B), and each of its 39 changes links to its condition.

CASEPATH is exact on every event after the claim; averaged over the insurer’s deadline, its declination and the recovery of stolen items, no baseline exceeds 0.40 (Table 3), and Direct never asks for the declination letter. CASEPATH’s chains explain its low scores on settlement and recourse: the model misread the expert-procedure request, the planner requested every route, and broad rules added two consents for a repair and kept the third party’s documents on both checklists. Three quarters of its errors lie in the rules the agents wrote (Appendix B.2).

Table 2: **CASEPATH makes the fewest wrong checklist changes and gets the most pairs exactly right.** The 27 held-out pairs require 33 changes, all additions; brackets: 95% intervals; bold: best.

Method	Checklist changes			Exact pairs ↑	F_1 [95% interval] ↑
	Correct ↑	Wrong ↓	Missed ↓		
CASEPATH	24	15	9	12/27	0.667 [0.435, 0.880]
Direct	25	27	8	9/27	0.588 [0.404, 0.759]
Graph context	31	41	2	4/27	0.590 [0.471, 0.716]
Evidence-first	16	52	17	2/27	0.317 [0.239, 0.400]

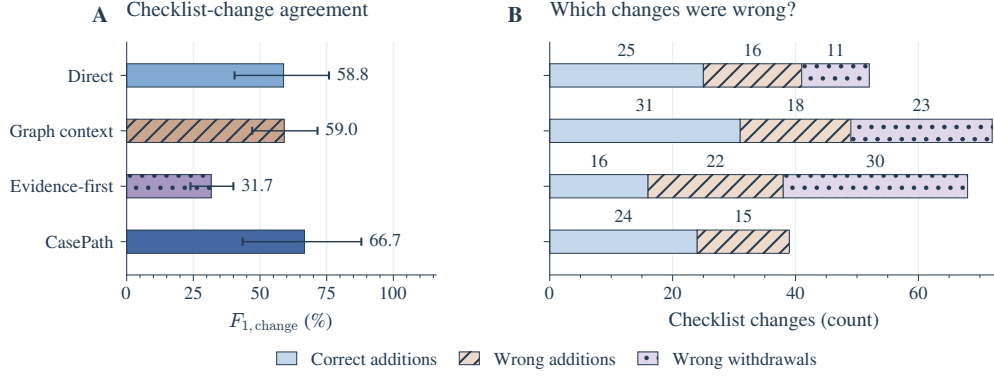


Figure 5: **The baselines’ extra errors are mostly withdrawals, which no pair requires.** (A) F_1 over all changes, 95% intervals. (B) Correct additions, wrong additions and wrong withdrawals.

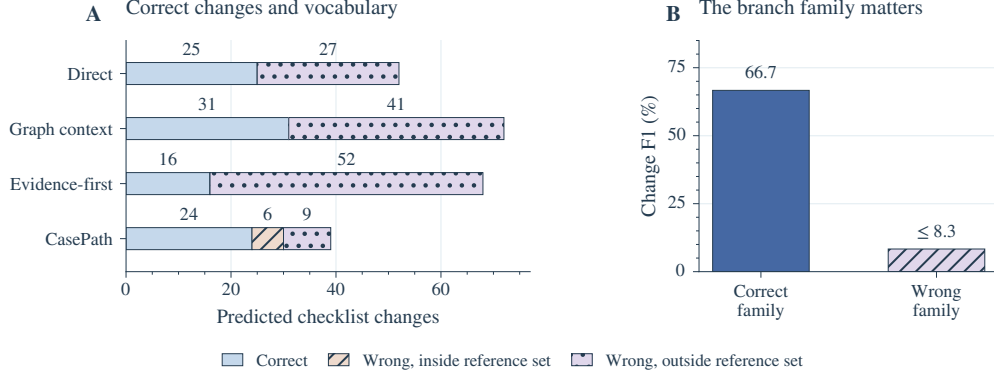


Figure 6: **The baselines change documents that the changed condition cannot justify.** (A) Correct and wrong changes; the reference set holds documents some branch governs. (B) CASEPATH’s F_1 with the correct condition (labelled family) and its best over 5,008 wrong pairings.

Table 3: **CASEPATH is exact on every event after the claim and weakest on settlement and recourse.** Mean pair F_1 over the three held-out stories of each condition; bold: best method.

	Branch condition	CASEPATH	Direct	Graph context	Evidence-first
Stolen property	Scheduled valuable	0.67	1.00	0.69	0.50
	Bicycle or e-bike	1.00	0.61	0.82	0.21
	Misused SIM card	0.67	0.78	0.61	0.39
Events after the claim	Insurer’s deadline	1.00	0.17	0.39	0.23
	Insurer’s declination	1.00	0.00	0.28	0.22
	Items recovered	1.00	0.80	0.52	0.39
Settlement and recourse	Formal expert procedure	0.00	0.49	0.44	0.56
	Repair needing consent	0.50	0.78	0.89	0.00
	Third party at fault	0.00	0.89	0.80	0.33
Mean over the nine conditions		0.65	0.61	0.60	0.31

On total error, CASEPATH is cheapest unless a miss costs more than $26/7 \approx 3.7$ wrong changes; above that, Graph context, which misses 7 fewer changes but makes 26 more wrong ones, is cheapest (Appendix Figure 10).

All five traceability and specificity criteria are met, and the three accuracy criteria are not: the margin over the closest baseline (0.076) and the precision of the changes (0.62) fall short of their targets, and

Table 4: CASEPATH meets five of eight predeclared criteria, none of them for accuracy.

	Criterion	Target	CASEPATH	Met
Accuracy	Smallest F_1 margin over a baseline	≥ 0.10	0.076	no
	Largest Holm-corrected paired p	≤ 0.05	0.797	no
	Precision of the changes	≥ 0.70	0.62	no
Traceability	Changes traced to a changed condition	≥ 0.90	1.00	yes
	Required changes reachable by a source chain	≥ 0.80	0.82	yes
	Requested documents with a source chain	all	all	yes
Specificity	Changes per required change	0.75–1.50	1.18	yes
	Correct- vs. wrong-condition F_1	>97.5 th pct.	0.667 vs. ≤ 0.083	yes

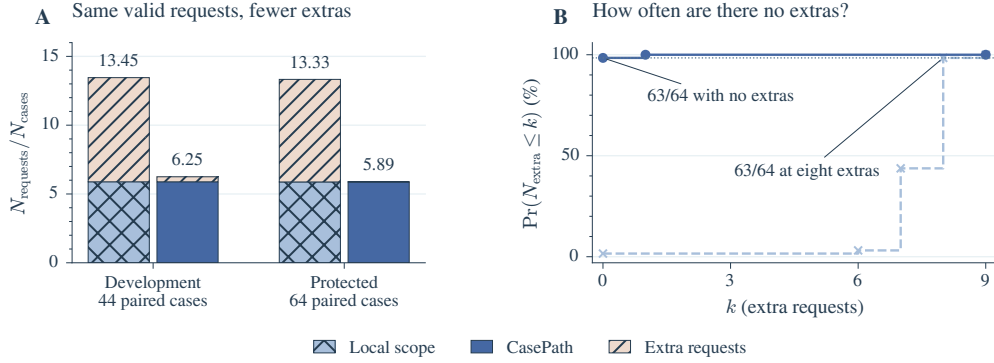


Figure 7: **Full process scope removes extra requests and keeps the valid ones.** (A) Requests per claim on the 108 claims planned both ways (64 held out, labelled protected); blue valid, peach extra. (B) Held-out claims with at most k extra requests.

Table 5: **Counting every claim, full process scope more than doubles justified-request precision and keeps coverage.** All 150 claims at the saved judgments; failed outputs score zero.

Method	Claims with output	Justified-request precision $\uparrow$	Critical-evidence recall $\uparrow$
CASEPATH	110 of 150	0.721	0.641
Local scope	108 of 150	0.323	0.630

the paired test over nine conditions finds no significant gain: against Direct, the losses on settlement and recourse offset most of the gains on the events after the claim (Table 4).

5.2 STUDY B: THE ENCLOSING PROCESS REMOVES REQUESTS THAT DO NOT APPLY

Full process scope cuts requests by more than half and keeps every valid one. On the 108 claims planned both ways, the inherited conditions cut requests from 1,445 to 652, and all 635 valid ones remained (Figure 7). By design, full scope can only remove or hold a request; the finding is that it removed no valid one. Every removed request belonged to another dispute type: 710 were held because the model left that type open, and 83 were dropped because it judged the type false. On 63 of the 64 held-out claims, CASEPATH made no extra request. Over all claims, justified-request precision rises from 0.323 under Local scope to 0.721, and coverage holds (Table 5).

Deleting as many requests at random would keep under half of the valid ones; full process scope keeps all of them (Figure 8B) and removes over half of the nodes and relations in the output graphs without losing a reference match (Figure 8A).

CASEPATH also produces a usable list far more often: its output, assembled by code, is scoreable for a weighted 73.6% of claims against at most 46.7% for the model-written baselines, and the lists that exist score similarly across pipelines (Appendix Figure 13). Conventional paired contrasts favour

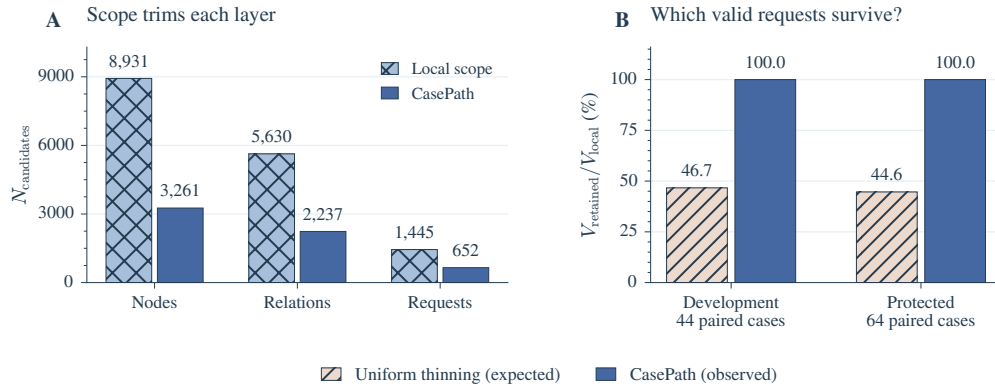


Figure 8: **Full process scope prunes every layer without losing a reference match.** (A) Nodes, relations and requests per scope. (B) Valid requests kept versus random deletion; protected: held out.

CASEPATH; the conservative ones, which count even a baseline’s failure against it, are all negative, so under the predeclared rule it beats no baseline (Appendix Table 13).

6 DISCUSSION

Process in the prompt is not process control. Graph context had CASEPATH’s graph and rules in its prompt and still made 41 wrong changes, more than Direct, all on documents no branch governs. Our results favour letting the process decide which actions are admissible and asking the model only for the judgments that code cannot make.

Information-seeking agents decide when to retrieve or what to ask; rule-governed work asks instead whether an active requirement needs the evidence now. When process state decides, each request has a reason, and an open fact, such as whether the watch was separately insured, becomes a question first. In Study B, CASEPATH held back 2,006 documents until a condition was resolved and the baselines rarely did (Appendix Table 12); no study scores whether a hold is right.

Limitations. Study A meets none of its three accuracy criteria, and under Study B’s predeclared rule CASEPATH beats no baseline. Study B’s evidence comes from rescoring saved runs, since the benchmark’s evaluator accepted none; its held-out claims were seen during development, and its process has one inherited condition. Of Study A’s nine misses, six sit in rules as the agents wrote them, and in three, requesting every route keeps the document on both checklists. CASEPATH separates reading from control, so the rules and the planner can each be improved on their own. We built both datasets, and neither study shows legal correctness in real cases; expert review, deeper processes, real claims and other jurisdictions come next.

7 CONCLUSION

CASEPATH enforces in code the process state that makes an action applicable; in both studies its requests were more selective, though not more complete, and each kept its reason. The released datasets³ let others train and test agents that ask what a case needs before requesting a document.

REPRODUCIBILITY STATEMENT

The anonymous supplement holds everything needed to check the results offline: for Study A, the frozen sources, rules, planner, benchmark, predictions, provider receipts and analysis code; for Study B, the 150 claims with their reference answers, the compiled process, the prediction code, the original evaluator, the document-request scorer and every recorded run, failures included. Each

³Datasets: <https://anonymous.4open.science/r/casepath-9673/data/>.

reported result is linked to its source file by hash, and the reproduction commands recompute every analysis without network or model access (Appendices B and C.5). Both datasets are also released on their own in the repository’s `data/`, each with its licence, datasheet and verification script (Appendix D). New model outputs would need provider access and are not deterministic.

ETHICS STATEMENT

The studies use public sources and synthetic cases that contain no personal data. The released datasets are synthetic apart from quoted law, policy and claim-form passages and official cantonal form templates; third-party text stays with its issuers and is excluded from the data licence. The evaluated Study B messages named a real insurer’s intake mailbox as their recipient; the release replaces it with a fictional address of the same byte length (Appendix D). CASEPATH is a research method for planning evidence requests; it gives no legal advice and makes no automated coverage decisions. Real use would require the correct policy and jurisdiction, professional review, privacy controls and ways to correct or appeal a decision.

AI USE STATEMENT

The authors conceived the research questions, designed the CASEPATH architecture and both studies, and directed the work throughout. In this work, we used generative AI tools as assistants for the following tasks with required disclosure: implementing, testing and running code; generating the synthetic cases of both datasets and Study A’s reference contract, including translating German source passages, to our specifications; reformatting and pseudonymizing the datasets for release; categorising Study A’s errors by their origin; and giving feedback on our framework, hypotheses, methodology and interpretation of results. We have not used them to formulate mathematical claims or to write proofs or their ingredients; no other task with required disclosure applies to this work. Additionally, we used generative AI tools for literature search, figures, and drafting and editing text. We have reviewed all AI-assisted work: code was tested, every reported number was recomputed from the released files, and every reference was checked against its source. We take responsibility for the final content of this work, including text, claims and artifacts produced with the aid of generative AI.

REFERENCES

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avi Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (eds.), *International Conference on Learning Representations*, volume 2024, pp. 9112–9141, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/25f7be9694d7b32d5cc670927b8091e1-Paper-Conference.pdf.
- AXA Versicherungen AG. Allgemeine Vertragsbedingungen (AVB) Haushaltversicherung: Privathaftpflicht, Hausrat, Cyber Plus, Zusatzversicherungen und Services. Policy terms, 16162DE, 2021. URL <https://www.axa.ch/servlets/external/docstoredocument?accesscode=ag58y>. Ausgabe 10.2021. Accessed 24 September 2026.
- David Basin, Søren Debois, and Thomas Hildebrandt. Proactive enforcement of provisions and obligations. *Journal of Computer Security*, 32(3):247–289, 2024. doi: 10.3233/JCS-210078.
- Sheng Bi, Kai Yu, Jiaqi Li, Xiaoyan Wang, Fuhui Sun, Zeyi Miao, Jing Wang, Xin Geng, and Lusheng Wang. CLER: A benchmark for Chinese litigation evidence reasoning. *Information Processing & Management*, 63(5):104667, 2026. doi: 10.1016/j.ipm.2026.104667. URL <https://www.sciencedirect.com/science/article/pii/S0306457326000592>.
- Samuel R. Bowman and George Dahl. What will it take to fix benchmarking in natural language understanding? In *Proceedings of NAACL-HLT*, pp. 4843–4855, 2021. doi: 10.18653/v1/2021.naacl-main.385.
- Zhaorun Chen, Mintong Kang, and Bo Li. ShieldAgent: Shielding agents via verifiable safety policy reasoning. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix

- Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 8313–8344. PMLR, 2025. URL <https://proceedings.mlr.press/v267/chen25ae.html>.
- Jiseong Chung, Ronny Ko, Wonchul Yoo, Makoto Onizuka, Sungmok Kim, Tae-Wan Kim, and Won-Yong Shin. GraphCompliance: Aligning policy and context graphs for LLM-based regulatory compliance. *arXiv preprint arXiv:2510.26309*, 2025. URL <https://arxiv.org/abs/2510.26309>.
- CSS Versicherung AG. Hausratversicherung: Allgemeine Versicherungsbedingungen (AVB). Policy terms, Form. 519d-01.24-eff, 2024. URL https://www.css.ch/content/dam/css/de/documents/privatkunden/richtig-versichert/avb/519_d_avb_hausratversicherung.pdf. Ausgabe 02.2024. Accessed 24 September 2026.
- CSS Versicherung AG. Schadenanzeige Hausrat / Gebäude. Claim notification form, Form. 15d-08.26-pdf, 2026. URL https://www.css.ch/content/dam/css/de/documents/privatkunden/richtig-versichert/formulare/15_d_formular_schadenanzeige_hausrat.pdf. Accessed 24 September 2026.
- Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3563–3578, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.212. URL <https://aclanthology.org/2024.findings-acl.212/>.
- Weihong Du, Wenrui Liao, Hongru Liang, and Wenqiang Lei. PAGED: A benchmark for procedural graphs extraction from documents. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10829–10846, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.583. URL <https://aclanthology.org/2024.acl-long.583/>.
- Robert Geirhos, Jørn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020. doi: 10.1038/s42256-020-00257-z.
- Generali Allgemeine Versicherungen AG. Allgemeine Versicherungsbedingungen (AVB) Hausratversicherung. Policy terms, 2026. URL <https://www.generali.ch/content/dam/generali/pdf/produkt dokumente/avb-ki/hausrat-haftpflicht/avb-haushaltversicherung-de.pdf>. Ausgabe 2026. Accessed 24 September 2026.
- David Graus. From legal text to executable decision models: Evaluating structured representations for legal decision model generation. *arXiv preprint arXiv:2604.17153*, 2026.
- Lin Guan, Karthik Valmeekam, Sarath Sreedharan, and Subbarao Kambhampati. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 79081–79094. Curran Associates, Inc., 2023. doi: 10.52202/075280-3459. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/f9f54762cbb4fe4dbffdd4f792c31221-Paper-Conference.pdf.
- Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *Proceedings of NAACL-HLT*, pp. 107–112, 2018. doi: 10.18653/v1/N18-2017.
- Dexter Hadley. CANONIC: Governance is compilation. *arXiv preprint arXiv:2607.05410*, 2026.
- Divyansh Kaushik and Zachary C. Lipton. How much reading does reading comprehension require? A critical investigation of popular benchmarks. In *Proceedings of EMNLP*, pp. 5010–5015, 2018. doi: 10.18653/v1/D18-1546.

- Onur Selim Kilic, Yeti Z. Gurbuz, Cem O. Yaldiz, Afra Nawar, Etrit Haxholli, Oğul Can, and Eli Waxman. Guideline2Graph: Profile-aware multimodal parsing for executable clinical decision graphs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pp. 6942–6951, June 2026. URL https://openaccess.thecvf.com/content/CVPR2026W/Med-Reasoner/html/Kilic_Guideline2Graph_Profile-Aware_Multimodal_Parsing_for_Executable_Clinical_Decision_Graphs_CVPRW_2026_paper.html.
- Myung Ho Kim. Evidence-obligation pool-gated retrieval: Stable multi-cycle retrieval via evidence ledger and warrant judge. Preprints.org, 2026. URL <https://www.preprints.org/manuscript/202607.1060>. Version 1, posted 15 July 2026; not peer reviewed.
- Dawei Li, Renliang Sun, Yue Huang, Ming Zhong, Bohan Jiang, Jiawei Han, Xiangliang Zhang, Wei Wang, and Huan Liu. Preference leakage: A contamination problem in LLM-as-a-judge. In *International Conference on Learning Representations*, 2026. arXiv:2502.01534.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of EMNLP*, pp. 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187.
- Shuyue Stella Li, Vidhisha Balachandran, Shangbin Feng, Jonathan S. Ilgen, Emma Pierson, Pang Wei Koh, and Yulia Tsvetkov. MediQ: Question-asking LLMs and a benchmark for reliable interactive clinical reasoning. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 28858–28888. Curran Associates, Inc., 2024. doi: 10.52202/079017-0908. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/32b80425554e081204e5988ab1c97e9a-Paper-Conference.pdf.
- Hugo A. López, Søren Debois, Tijs Slaats, and Thomas T. Hildebrandt. Business process compliance using reference models of law. In *Fundamental Approaches to Software Engineering*, pp. 378–399, 2020. doi: 10.1007/978-3-030-45234-6_19.
- Andrew Lou and David Shin. Legal reasoning is not lawyering: Rethinking legal benchmarks for pro se access to justice. *arXiv preprint arXiv:2606.23716*, 2026.
- Yuxing Lu, Yicheng Chen, Shanchan Wu, and Sercan Ö. Arık. Procedural graphs: Self-evolving execution structures for LLM agents. *arXiv preprint arXiv:2609.09153*, 2026.
- R. Thomas McCoy, Ellie Pavlick, and Tal Linzen. Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In *Proceedings of ACL*, pp. 3428–3448, 2019. doi: 10.18653/v1/P19-1334.
- Masha Medvedeva and Pauline McBride. Legal judgment prediction: If you are going to do it, do it right. In *Proceedings of the Natural Legal Language Processing Workshop 2023*, pp. 73–84, 2023. doi: 10.18653/v1/2023.nllp-1.9.
- Jacob Menick, Maja Trebacz, Vladimir Mikulik, John Aslanides, Francis Song, Martin Chadwick, Mia Glaese, Susannah Young, Lucy Campbell-Gillingham, Geoffrey Irving, and Nat McAleese. Teaching language models to support answers with verified quotes. *arXiv preprint arXiv:2203.11147*, 2022. URL <https://arxiv.org/abs/2203.11147>.
- Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: A programming language for the law. *Proceedings of the ACM on Programming Languages*, 5(ICFP):1–29, 2021. doi: 10.1145/3473582. URL <https://arxiv.org/abs/2103.03198>.
- OASIS LegalRuleML Technical Committee. LegalRuleML core specification version 1.0. OASIS Standard, 2021. URL <https://docs.oasis-open.org/legalruleml/legalruleml-core-spec/v1.0/os/legalruleml-core-spec-v1.0-os.html>.
- Object Management Group. Case management model and notation (CMMN), version 1.1. OMG Specification, 2016. URL <https://www.omg.org/spec/CMMN/1.1>.
- Object Management Group. Decision model and notation (DMN), version 1.5. OMG Specification, 2024. URL <https://www.omg.org/spec/DMN/1.5>.

- Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 3806–3824, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.248. URL <https://aclanthology.org/2023.findings-emnlp.248/>.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- Adam Poliak, Jason Naradowsky, Aparajita Haldar, Rachel Rudinger, and Benjamin Van Durme. Hypothesis only baselines in natural language inference. In *Proceedings of *SEM*, pp. 180–191, 2018. doi: 10.18653/v1/S18-2023.
- Shuofei Qiao, Runnan Fang, Zhisong Qiu, Xiaobin Wang, Ningyu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. Benchmarking agentic workflow generation. In *International Conference on Learning Representations*, volume 2025, pp. 69679–69703, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/adbe936993aa7cf41e45054d8b72f183-Abstract-Conference.html.
- Schweizerischer Versicherungsverband SVV. Allgemeine Bedingungen (AVB) für die Hausratversicherung. Non-binding model policy conditions (unverbindliche Musterbedingungen), Ausgabe 2022, 2022. URL <https://svv.ch/sites/default/files/media/documents/2024-02/AVB%20Hausratversicherung%202022.pdf>. Version 01.04.2022. Accessed 24 September 2026.
- ServiceNow. Add a document list definition to a service definition. Financial Services Operations documentation, 2026. URL <https://www.servicenow.com/docs/r/financial-services-operations/insurance-claims/add-document-list-definition-to-service-definition.html>. Brazil release; updated September 10, 2026. Accessed 24 September 2026.
- Simpego Versicherungen AG. Simpego Haushaltversicherung Home: Allgemeine Versicherungsbedingungen (AVB). Policy terms, 2024. URL https://simpego.ch/dam/AVB_Simpego_Home_de.pdf. Ausgabe Dezember 2024. Accessed 24 September 2026.
- Saku Sugawara, Kentaro Inui, Satoshi Sekine, and Akiko Aizawa. What makes reading comprehension questions easier? In *Proceedings of EMNLP*, pp. 4208–4219, 2018. doi: 10.18653/v1/D18-1453.
- Swiss Confederation. Bundesgesetz vom 2. April 1908 über den Versicherungsvertrag (Versicherungsvertragsgesetz, VVG), SR 221.229.1. Fedlex, Swiss Federal Chancellery, 2024. URL https://www.fedlex.admin.ch/eli/cc/24/719_735_717/20240101/de. Consolidated version of 1 January 2024 (Stand am 1. Januar 2024), German text; Arts. 38–41a, 45 and 46 used. Accessed 24 September 2026.
- Swiss Confederation. Bundesgesetz vom 30. März 1911 betreffend die Ergänzung des Schweizerischen Zivilgesetzbuches (Fünfter Teil: Obligationenrecht), SR 220. Fedlex, Swiss Federal Chancellery, 2026. URL https://www.fedlex.admin.ch/eli/cc/27/317_321_377/20260101/de. Consolidated version of 1 January 2026 (Stand am 1. Januar 2026), German text; tenancy provisions Arts. 257d–273 used. Accessed 24 September 2026.
- Maksym Taranukhin, Shuyue Stella Li, Evangelos Milios, Geoff Pleiss, Yulia Tsvetkov, and Vered Schwartz. InfoGatherer: Principled information seeking via evidence retrieval and strategic questioning. *arXiv preprint arXiv:2603.05909*, 2026. URL <https://arxiv.org/abs/2603.05909>.

- Paul Thomas, Nick Craswell, Mark Sanderson, Seth Spielman, Robert Sim, and Ryen W. White. Evaluation validity in information retrieval. In *Proceedings of SIGIR*, pp. 2734–2746, 2026. doi: 10.1145/3805712.3808538.
- Roman Vaculín, Richard Hull, Terry Heath, Craig Cochran, Anil Nigam, and Piyawadee Sukaviriya. Declarative business artifact centric modeling of decision and knowledge intensive business processes. In *2011 IEEE 15th International Enterprise Distributed Object Computing Conference*, pp. 151–160, 2011. doi: 10.1109/EDOC.2011.36.
- Yiran Wu, Tianwei Yue, Shaokun Zhang, Chi Wang, and Qingyun Wu. StateFlow: Enhancing LLM task-solving through state-driven workflows. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=3nTbuygoop>.
- Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 68316–68342. PMLR, 2025. URL <https://proceedings.mlr.press/v267/xiang25a.html>.
- Ruixuan Xiao, Wentao Ma, Ke Wang, Yuchuan Wu, Junbo Zhao, Haobo Wang, Fei Huang, and Yongbin Li. FlowBench: Revisiting and benchmarking workflow-guided planning for LLM-based agents. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 10883–10900, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.638. URL <https://aclanthology.org/2024.findings-emnlp.638/>.
- Chen Yang, Ruping Xu, Ruizhe Li, Bin Cao, and Jing Fan. Business as rulesual: A benchmark and framework for business rule flow modeling with LLMs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 35198–35220. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1625.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (eds.), *International Conference on Learning Representations*, volume 2025, pp. 9965–10017, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/1b126cc38b8638e07bef37e7b2bb72bf-Paper-Conference.pdf.
- Chenglin Yu, Li Yin, Qingxin Fan, Ying Yu, Runyang Ray Zhong, and Ming Li. Compile, then page: Executable SOP programs and a capability-gated runtime for procedural LLM agents, 2026. URL <https://arxiv.org/abs/2607.11346v3>. Version 3, 23 July 2026.
- Yu Yuan, Lili Zhao, Kai Zhang, Guangting Zheng, and Qi Liu. Do LLMs overcome shortcut learning? An evaluation of shortcut challenges in large language models. In *Proceedings of EMNLP*, pp. 12188–12200, 2024. doi: 10.18653/v1/2024.emnlp-main.679.
- Xuan Zhang, Yang Deng, Zifeng Ren, See-Kiong Ng, and Tat-Seng Chua. Ask-before-Plan: Proactive language agents for real-world planning. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 10836–10863, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.636. URL <https://aclanthology.org/2024.findings-emnlp.636/>.

A HOW THE TWO STUDIES IMPLEMENT THE METHOD

Both studies implement the idea of Section 3: work out which requirements are active for the case, then request only the evidence they need. Study A uses the smallest controller that can respond to a changed branch condition, in which each rule checks only its own condition. Study B’s planner implements the full design, adding conditions inherited from enclosing branches, the timing of a request, the adequacy of documents on file and the choice of one route. Table 6 sets the two side by side. The studies share no sources, cases or code, and we do not compare their planners.

Table 6: **The two studies share the process-first idea but not their code, cases or measurements.**

	Study A	Study B
<i>Method</i>		
Rules	Agents turn law and policy texts into local evidence rules mapped to document routes	Code compiles three synthetic tenancy templates, shared by every method
What activates a request	The rule’s own condition; an unresolved one becomes a question	The obligation’s own condition and the dispute type above it; an unresolved one becomes a question
Evidence on file	Held document types are left out (none were held); every route is requested	Each document is checked for presence and adequacy; one single-document route per capability
When the case changes	The plan is recomputed from the edited case	The plan is recomputed from a new assessment
<i>Evaluation</i>		
Cases	9 development and 27 held-out pairs over nine branch conditions; 72 cases	60 development and 90 held-out claims from 11 and 17 families covering defects, terminations and rent increases
Model calls	One call per condition over all cases of a split for CASEPATH; one call per case for Direct and Graph context, two for Evidence-first	Two per claim for each of five pipelines; two controls reuse CASEPATH’s assessments
What is scored	Checklist changes between the two cases of a pair	Complete outputs (full-output evaluation) and requested documents (document-request scoring)
Statistics	Bootstrap over conditions, exact paired tests and a wrong-condition control; the three accuracy criteria fail	Weighted descriptive contrasts; no significance claim
Released workbench	Replaying the recorded decisions reproduces all 72 cases and the changes of all 36 pairs	Separate implementation; no parity claim

Study A’s planner requests every route of every active rule. Study A’s rules play the role of obligations. A rule is active when the case meets its condition, or always if it has none; a rule whose condition is unresolved yields a question instead. For each evidence capability an active rule needs, the planner requests every route, together with the routes that would confirm each condition judged true. It leaves out document types already on file without checking their adequacy (no Study A case has documents on file), and the larger process graph plays no part. Formally, let $A(x)$ be the active rules of case x , C_r the evidence capabilities of rule r , $\mathcal{R}_c$ the document routes of capability c , $\mathcal{R}_g^{\text{conf}}$ the routes that confirm condition g , and H the document types already held. The planner requests

$$D(x, H) = \left[\bigcup_{r \in A(x)} \bigcup_{c \in C_r} \bigcup_{\rho \in \mathcal{R}_c} \rho \cup \bigcup_{g \text{ true}} \bigcup_{\rho \in \mathcal{R}_g^{\text{conf}}} \rho \right] \setminus H. \quad (1)$$

Since every route contributes, the planner never chooses between alternatives, and a rule without a condition puts all of its routes on every checklist. This is how the experts’ appointment, one route of the rule on determining the loss, appears in both cases of every pair (Table 8). Even if the valuable’s rule had listed the receipt and the expert valuation as two routes, the planner would have requested both. Each request keeps its rule, required fact, evidence capability and source quotation. Every correction to the rules was made from the sources alone and stays in the record.

Study B’s planner adds inherited conditions, timing and route choice. For each obligation, the planner decides whether it applies, whether its evidence may be requested yet, and which set of documents should supply that evidence. In Figure 2, the obligation to assess and address the mould applies only if its own condition holds (the landlord has been told) and so does every branch above it. In the model’s assessment z , each condition is true, false or unresolved. For a branch s with condition ϕ_s , and an obligation o in branch $\text{scope}(o)$ with local condition ϕ_o and timing condition ψ_o , the planner computes

$$\begin{aligned} q_s(z) &= \phi_s(z) \wedge_3 J_s(\{q_p(z)\}_{p \in \text{pa}(s)}), \\ a_o(z) &= q_{\text{scope}(o)}(z) \wedge_3 \phi_o(z), \quad b_o(z) = a_o(z) \wedge_3 \psi_o(z). \end{aligned} \quad (2)$$

The first line checks a branch and, through its parents $\text{pa}(s)$, every branch above it; J_s says whether all or any of the parents must apply, and a root has no parent requirement. The second line decides whether the obligation applies (a_o) and whether its evidence may be requested now (b_o). The three-valued conjunction $\wedge_3$ is false if any input is false, true if all inputs are true, and unresolved otherwise. The planner requests an obligation’s documents now only when b_o is true and drops them when a_o is false; otherwise it holds them back and, if a condition is unresolved, asks about it (Appendix C.3). The Local scope control of Section 4.2 cuts every link to the branches above and keeps everything else, so each obligation checks only its own condition and that of its branch. Because $\wedge_3$ can only lower a value, full scope can remove or defer a request that Local scope makes and can never add one.

Each obligation o needs evidence capabilities $\mathcal{C}_o$ (written $\mathcal{C}_r$ for a Study A rule), and capability c establishes the required fact f_c . In the evidence state E , the model judges whether each document on file is present and adequate. A route satisfies a capability ($\rho_c(E) = \text{T}$) only if its documents are present, adequate and sufficient together. Let M_{cp} be the route’s documents that are missing or inadequate, and V_{cp} the items whose presence or adequacy needs review. For each capability whose evidence may be requested, the planner chooses

$$\rho_c^* = \arg \min_{\rho \in \mathcal{R}_c}^{\text{lex}} (\mathbf{1}\{\rho_c(E) \neq \text{T}\}, |M_{cp}|, |V_{cp}|, \text{sort}(M_{cp}), \text{id}(\rho)) \quad (3)$$

and requests $\bigcup_c M_{cp_c^*}$. It prefers a route the file already satisfies, then the route with fewer missing or inadequate documents, then the one with fewer items to review; sorted document identifiers and the route identifier break ties. Routes are chosen one capability at a time, and a capability without any route is recorded as an evidence gap. The next action follows a fixed order: ask a question, review evidence, request documents, review an open gap, and only then take a permitted case action.

Each Study B obligation inherits one condition, the dispute type. Code compiles the process deterministically from the three tenancy templates, and every method uses the result. The only condition an obligation inherits from the branches above it is the claim’s dispute type (a defect, a lease termination or a rent increase), which the model judges from the case. Every timing condition ψ_o is true, and each capability has a single route with one document. Study B therefore tests inherited applicability, the deferral of unresolved branches and the state of documents on file. The timing and route rules run with nothing to decide, and a deeper process would give them real choices. Even so, this one inherited condition removes more than half of the requests (Section 5.2).

A.1 THE RELEASED SOFTWARE

CASEPATH is also being built into claims-handling software. The software is under active development; the released version is a local, single-user workbench. A browser interface runs over a service that stores every change to a claim as an event in a hash-chained journal. A claims handler opens any of the 150 claims of Study B and reads the original message and attachments beside the active process step, the evidence to check and the next action. The handler can then record, correct, export and replay the handling state; nothing is sent to a customer. The workbench’s own review places a claim in one of the three process templates and runs six agent roles in a fixed order, of which only the process role may change claim state. In the released version this review is deterministic and makes no model calls. It is product behaviour and inherits none of the results of Section 5. Table 7 separates the method, the benchmarks and the product.

Given the same assessments, the software makes Study A’s decisions. A verifier loads the installed method pack and the service and replays all 72 cases with the recorded condition decisions.

Table 7: **The paper measures the method; the software hosts it.** Where each part of the release lives and what supports it.

Part	What it is	Evidence
Study A method	Condition interpreter and planner of Equation 1, installed in the service as a separate endpoint	Offline replay of all 72 cases matches
Study B method	Controller and evidence planner of Equations 2 and 3; shown on an authored teaching example	Section 5.2; 45 teaching states re-executed
Benchmarks	Study A scorer and explorer; Study B evaluator, document-request scorer and state-change track	Appendices B and C
Workbench	Interface, journal, agent review, correction and replay over the 150 claims	Demonstrates handling mechanics only

It compares verdicts, document sets and next actions, checks all 36 paired changes with the service’s comparison function, and confirms that every displayed quotation occurs exactly in its source passage. Every check passes. A live run of the service needs provider access and covers theft claims only. A copy downloaded from the anonymous mirror does not start the workbench, because the mirror rewrites strings in files that the launcher checks; the dataset verifiers and the Study A replay are unaffected.

B STUDY A IN DEPTH

Study A changes one fact at a time, and each of CASEPATH’s requests keeps its chain, so every error it makes can be followed to where it arose.

B.1 BENCHMARK AND SCORING

Every extracted statement has a rule or needs no evidence. The agents extracted 308 statements from 197 passages drawn from Swiss federal law, four insurers’ policy terms, a claim form and industry model conditions, and either covered each with a rule or marked it as requiring no evidence. The frozen representation holds 98 evidence rules, 13 statements that require no evidence, 87 case conditions and 33 document types. Completeness is judged against these statements; other law that could apply is out of scope. The nine branch conditions, grouped as in Table 3, are: *stolen property*: a scheduled valuable at or above the threshold defined in the sources, a stolen bicycle or e-bike, and theft of a SIM card with a claim of misuse; *events after the claim*: a written demand with a warning of forfeiture and a deadline, the insurer’s declination, and the recovery of stolen items or news of them; *settlement and recourse*: the choice of a formal expert procedure, a planned repair above the consent threshold, and an identifiable third party who caused the loss. Each condition appears in four neutral stories, and within a pair only the branch statement changes. The release gives each passage verbatim, and its source notes give the URL and file hash of each document (Appendix D.1).

Each pair is scored against its closest acceptable reference. For a pair (x^-, x^+) whose branch condition is off in x^- and on in x^+ , a system’s signed change collects the documents it adds and withdraws:

$$\Delta(D) = \{+d : d \in D(x^+) \setminus D(x^-)\} \cup \{-d : d \in D(x^-) \setminus D(x^+)\}. \quad (4)$$

When the sources allow alternatives, a pair has several acceptable references $G \in \mathcal{G}_i$, one for each combination of alternatives. The scorer compares the prediction $P_i = \Delta(D_i)$ with each of them and keeps the one that maximizes, in lexicographic order,

$$(F_1(P_i, G), |P_i \cap G|, \text{Prec}(P_i, G), \text{Rec}(P_i, G), -|G|, \text{sort}(G));$$

the last entry breaks any remaining tie. Each reference holds one alternative, so asking for both when one suffices counts as a wrong change. Precision and recall are zero when their denominators are empty, and F_1 is one when both the prediction and the reference are empty. Table 2 pools the counts over the 27 held-out pairs, and Table 3 averages pair F_1 within each condition. Every reference change is an addition, so the benchmark rewards a system that withdraws nothing and cannot test whether it withdraws a document correctly.

B.2 WHERE THE ERRORS COME FROM

Most errors of CASEPATH sit in rules that can be corrected once. CASEPATH makes 15 wrong changes and misses 9 required ones, all on five of the nine conditions; the bicycle and the three events after the claim have none. Table 8 locates each one through the rule behind the request and the quotation behind the verdict. Eighteen of these 24 errors lie in the rules as the agents wrote them, where each can be found through its chain and corrected once. In nine, a condition is broader than the reference and the model’s verdict follows its wording. The condition on preserving evidence covers any planned alteration of damaged property, so a planned repair adds two consents; the condition on recourse covers any third party, so the unidentified thief keeps the assignment of claims on both checklists. In the other nine, a route or rule asks for too much: one route joins two alternatives, the SIM rule’s only route adds a bank statement, and one rule has no condition at all. Three errors come from misreading the case and three from the planner’s simplest policy of requesting every route. The nine misses have one mechanism in common: each missed document was already on the checklist before the condition changed, so the change had nothing to add.

Table 8: **Three quarters of CASEPATH’s errors in Study A trace to rules as the agents wrote them.** Wrong and missed changes over the three held-out stories of each condition, by where the error arose.

Condition	Wrong	Missed	What happened
<i>A condition broader than the reference</i>			
Repair needing consent	6	0	The condition covers any planned alteration of damaged property, so a repair adds the insurer’s and the police’s consent
Third party at fault	0	3	The condition covers any third party, so the unidentified thief keeps the assignment of claims on both checklists
<i>A route or rule that asks for too much</i>			
Scheduled valuable	3	0	One route asks for the receipt and the expert valuation together, where the sources accept either
Misused SIM card	3	0	The rule’s only route adds a bank statement to the theft report
Third party at fault	0	3	The claim-form section on the liable person comes from a rule without a condition, so it is on both checklists
<i>A misread case</i>			
Formal expert procedure	3	0	A written request for the procedure is read as one to appoint an expert, which adds the insurer’s written demand
<i>The planner</i>			
Formal expert procedure	0	3	Requesting every route keeps the experts’ appointment on both checklists
Total	15	9	

The baselines change documents that no branch governs. The *reference vocabulary*, labelled reference set in Figure 6, holds every document that some branch of the benchmark governs: the documents in all reference annotations and in every acceptable signed change. Figure 9 splits each method’s changes by this vocabulary for every condition. The baselines change documents outside it on almost every condition, adding or dropping general claim documents such as the claim form or the payment details. Each baseline writes the checklist of each case afresh, so nothing holds the unaffected documents fixed across a pair; CASEPATH derives both checklists from the assessed conditions through the same rules. Its nine changes outside the vocabulary are the repair consents and the SIM bank statement of Table 8, and its other six wrong changes use documents that the branches govern. The split reuses the reference the scorer chose for each pair, so no score changes.

The break-even cost of a miss rests on one condition. Figure 10 weighs the two kinds of error in Table 2. With a wrong change costing one and a missed change costing $\lambda \geq 0$, the total costs are

$$\begin{aligned} L_{\text{CasePath}} &= 15 + 9\lambda, & L_{\text{Direct}} &= 27 + 8\lambda, \\ L_{\text{Graph}} &= 41 + 2\lambda, & L_{\text{Evidence}} &= 52 + 17\lambda. \end{aligned}$$

CASEPATH is cheapest for $\lambda < (41 - 15)/(9 - 2) = 26/7$ and Graph context above that point; the other two methods are never cheapest. Six of CASEPATH’s nine misses come from the third party at

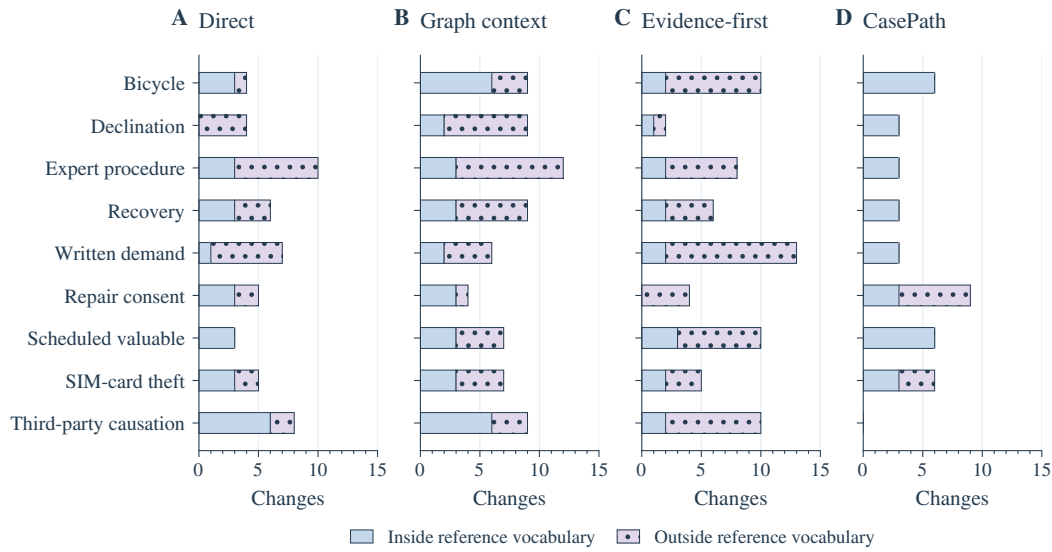


Figure 9: **The baselines leave the reference vocabulary on almost every condition; CASEPATH does so on two.** Changes by method and branch condition over the three held-out stories of each. A change inside the vocabulary can still be wrong, in its sign or its branch.

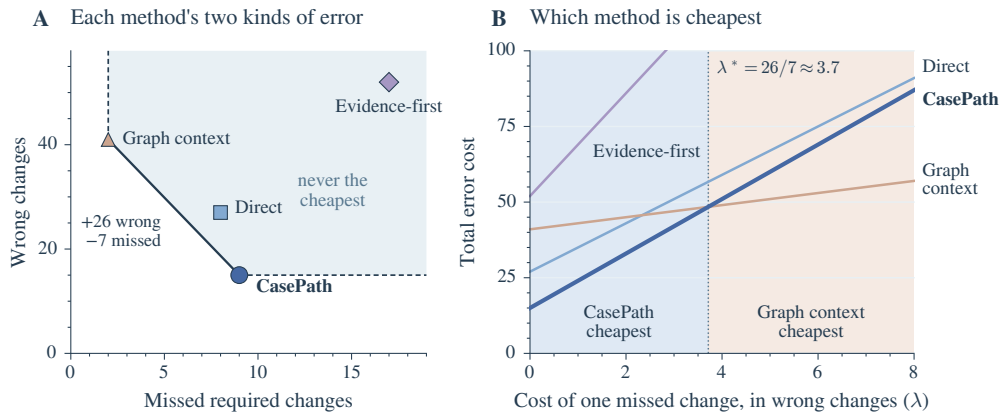


Figure 10: **CASEPATH has the lowest total error unless a missed change costs more than 3.7 wrong ones.** (A) Wrong against missed changes; methods in the shaded region are never the cheapest. (B) Total error when a wrong change costs 1 and a missed change λ .

fault, whose two required documents were already on both checklists. Without those six, and with the other counts unchanged, the crossing point would move from 3.7 to 26. The calculation prices all documents alike and uses observed counts without an uncertainty estimate.

B.3 STATISTICS AND PREDECLARED CRITERIA

Everything was frozen before the held-out read. One hash-bound freeze covered the sources, the benchmark with its two amendments made before evaluation, the extracted rules, the document map, the condition interpreter and the planner. It also covered the model, provider and temperature settings, the analysis code, the development split, the baselines' development predictions and the method's tests. The baselines are our own implementations, not reproductions of published systems. Development used 9 pairs (18 cases), one story per condition. Once predeclared checks on these pairs passed, we read the other 27 pairs (54 cases) once, under a separate freeze. The eight criteria of

Table 4 were fixed before that read, and no file changed after it. The criteria were not filed with an external registry; they are part of the hash-bound freeze in the supplement.

With nine conditions, the paired test needs consistent gains. The 95% intervals come from 10,000 bootstrap draws that resample the nine conditions with replacement (seed 20260917), each keeping its three held-out stories. The exact one-sided paired test enumerates all 512 sign assignments of the nine per-condition differences; its p -values against Direct, Graph context and Evidence-first are 0.436, 0.398 and 0.043, and 0.797, 0.797 and 0.129 after Holm correction. Against Direct, the losses on settlement and recourse offset most of the gains on the events after the claim.

The changes depend on the case. An input-free control fits the best constant change on eight conditions and scores it on the ninth; its highest F_1 is 0.0, so a fixed answer learned from other conditions scores nothing. The wrong-condition control of Figure 6B reassigns the saved changes to other conditions by 8 cyclic shifts and 5,000 random derangements, with the add-one p -value $(1+N_{\geq})/(1+5,008)$, where $N_{\geq}$ counts reassignments scoring at least the observed F_1 . The control’s reported maximum covers only these reassignments. A lookup from conditions to documents would pass both controls, which is why Table 3 and Table 8 examine the changes themselves.

B.4 COST

CASEPATH’s calls all go to judging the case. Table 9 lists the provider receipts of the held-out run, and Figure 11 sets them against quality. CASEPATH spends every call on assessing conditions and leaves the checklist to code. Each call judges one condition for all 54 held-out cases at once; the baselines read one case per call. CASEPATH reads fewer tokens than Direct and writes about five times as many. Among the baselines, prompt size explains most of the differences: Evidence-first reads only twelve retrieved passages, while Graph context reads the process graph and the rules on top of the sources.

Table 9: **CASEPATH costs more than Direct and less than Graph context, with more calls than either.** Held-out run; every method uses `openai/gpt-5.6-terra` at temperature 0 with one logged attempt per call. Max completion is the token limit per call; Evidence-first allows 18,000 tokens for extraction and 14,000 for planning. Charges exclude source preparation.

Method	Calls	Prompt tokens	Completion tokens	Max completion	Cost (USD)
CASEPATH	87	1,005,211	243,891	10,000	5.44
Direct	54	2,900,625	45,678	14,000	4.83
Graph context	54	5,345,421	42,663	14,000	8.41
Evidence-first	108	259,890	66,450	18,000/14,000	1.06

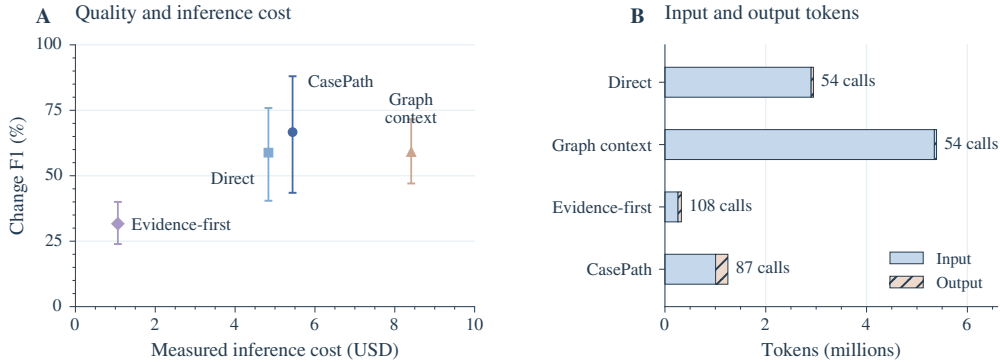


Figure 11: **CASEPATH has the highest change F_1 at a cost between Direct and Graph context.** (A) Provider charge and change F_1 with 95% intervals. (B) Input and output tokens, with the number of calls.

C STUDY B IN DEPTH

Study B holds the model’s judgments fixed and asks what the enclosing process adds. The benchmark’s own evaluator scored no output, so the evidence comes from controlled analyses of the saved runs.

C.1 RUNS, FAILURES AND THE ORIGINAL EVALUATOR

Every run stays in the record. A run is one method applied to one claim, so five pipelines and two controls on 150 claims give 1,050 runs (the cells of Figure 12). Each pipeline gets 2 model calls per claim, with medium reasoning effort, 4,096 completion tokens per call, no tools and no resampling. The 90 held-out claims appear as *protected* in the figures. A run records either a validated output or the provider’s output with the reason it failed validation. Most failures are of five kinds: a relation points to a node the output does not contain; a completion reaches the token limit or ends without a final answer; an index does not resolve in the shared public codebook; a document’s presence contradicts its assessed state; or a control is blocked because the CASEPATH run it depends on failed. The rest fail other checks, such as an incomplete assessment or a document outside the assessed route. The four baselines must write the whole output in two calls and fail on more than half of their claims; CASEPATH, whose output is assembled by code, fails on 40 (Figure 12A). Every failure keeps its charges, and no repaired output replaces a failed run.

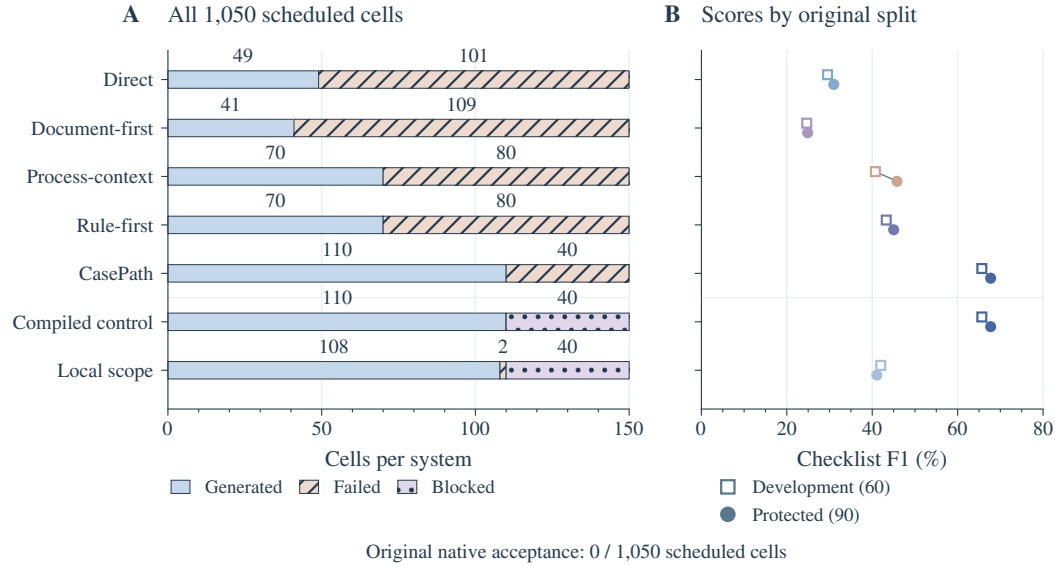


Figure 12: **Every run stays in the record, and scores are similar on both splits.** (A) Generated, failed and blocked runs per method, with the controls below the line; the benchmark’s evaluator accepted none. (B) Checklist F_1 from document-request scoring by split, with failed runs scored as zero.

The original evaluator rejected every output. The benchmark’s evaluator accepts an output only after evaluating its conditions against the reference scenario. Every output uses only the 50 condition symbols that the public compiler introduces for all methods, but the reference scenarios do not assign them. All 558 generated outputs therefore fail the check, all 1,050 runs receive the failure penalty, and none of the 12 predeclared targets for the held-out claims is met. Renaming the symbols would not fix this, because the two sides read an absent condition differently. In a counterexample written without looking at any target, CASEPATH’s controller returns unresolved for an absent condition and the evaluator’s parser raises a missing-variable error; with null substituted, the evaluator returns false while the controller still returns unresolved. Renaming a symbol, supplying a default or injecting a predicted state would each silently turn “not yet known” into “false”, so neither analysis below does any of these.

C.2 HOW THE REQUESTS ARE SCORED

Requests are scored by their chains and by their documents. *Justified-request precision* is the share of requested documents whose chain from obligation to required fact, evidence capability and document matches the reference. It equals one for an empty list, so it is read together with coverage. *Critical-evidence recall*, or coverage, weights each required piece of evidence by its importance and counts it only when a complete route is present for an active obligation. *Document-request scoring* compares the distinct documents P that a pipeline requests now with the reference documents G , through the checklist $F_1 = 2|P \cap G|/(|P| + |G|)$ and the unnecessary fraction $U = |P \setminus G|/|P|$; it never uses the reference state to decide which predicted conditions apply. When both sets are empty, F_1 is one; an empty prediction has precision one and an unnecessary fraction of zero, and an empty reference has recall one.

Failed outputs keep the worst score. The predeclared procedure scores a request list only after the evaluator accepts the whole output, so no list was scored. Document-request scoring changes only this eligibility rule. It reads the unchanged list of documents each output requests now, maps their labels through the original public mapping and applies the original scorer, without evaluating any condition against the reference. Each of the 150 references provides one eligible set of documents. The 558 generated lists receive observed scores, and the 492 failed or blocked outputs keep the frozen worst case: zero where higher is better and one where lower is better. We invent no requests for missing outputs, and a missing reference label never shrinks a denominator.

Domains count equally, and contrasts compare two outputs for the same claim. Claim i has weight $\omega_i = 1/(3m_{d(i)}n_{f(i)})$, where m_d is the number of families in its domain and n_f the number of claims in its family, so the three domains count equally and so do the families within a domain. In Figure 13, availability A is the weighted share of claims with a scoreable list and quality Q the weighted mean F_1 over those claims, so overall $F_1 = AQ$. A conventional paired contribution of claim i against method a is $b_m(s_{i,CP,m} - s_{i,a,m})$, where $b_m \in \{-1, 1\}$ is the benefit direction of metric m ; a conservative one uses this value when both scores exist and -1 otherwise. Under the predeclared rule, a baseline counts as beaten only if the conservative contrasts in checklist F_1 , critical-evidence recall and the unnecessary fraction all exceed their thresholds over all held-out claims.

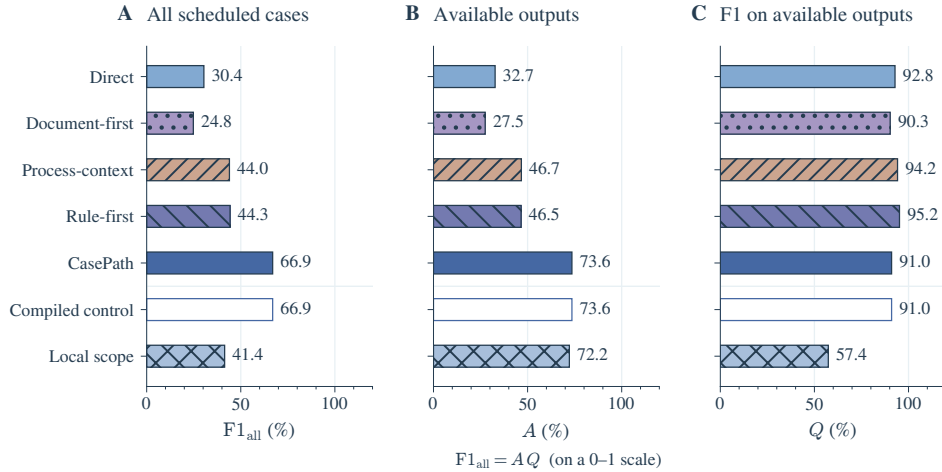


Figure 13: **CASEPATH produces a usable list more often; when lists exist, all pipelines do similarly well.** Over all 150 claims: (A) checklist F_1 with failures scored as zero, (B) weighted share A of claims with a scoreable list, (C) average $F_1 Q$ of those lists; overall F_1 is AQ .

C.3 WHAT FULL PROCESS SCOPE REMOVES, DEFERS AND KEEPS

Activation never sees the reference. The analysis of Section 5.2 evaluates every original condition against the fixed saved assessment, never against the reference. Entities known to be active enter

a snapshot of the current case; unresolved ones stay on record but inactive. Conditions follow the three-valued logic of Appendix A, where negation swaps true and false and disjunction is dual to $\wedge_3$. Branch conditions that describe other states of the case are not evaluated, and neither is the evaluator’s acceptance check on the whole output. The evaluator’s reference graphs, scoring functions and matching rules stay unchanged. Each control keeps its own graph and requests, bound to its saved CASEPATH assessment. All 110 snapshots and scores of Compiled control match CASEPATH.

Full scope removes only unnecessary requests, and defers most of them. On the 108 claims planned both ways, Local scope asks for 1,445 documents now and full scope for 652. On every claim, both ask for the same 635 correct documents and miss the same 93 of the 728 required ones. The whole difference is in unnecessary requests, which fall from 810 to 17, and every removed request belongs to one of the other two dispute types (Appendix A). Where the model leaves such a type unresolved, full scope holds its documents back until that question is answered (710 requests on 95 claims). Where the model judges it false, full scope drops its documents, whether Local scope requested them now or held them (83 and 103 on 11 claims). Held documents therefore rise from 1,387 to 1,994 on these claims; over all 110 of its lists, CASEPATH holds back 2,006. Because full scope can only remove or defer a request, it cannot raise coverage, which depends on the assessments and on what the process asks for. Requests fall from 592 to 275 on development claims and from 853 to 377 on held-out claims, and every valid request stays (259 and 376). Pooled over both splits, candidate nodes fall from 8,931 to 3,261 and relations from 5,630 to 2,237, while the reference matches for nodes (2,331), relations (2,031) and requests (635) stay the same on every claim. These totals weight every item equally; the rates over all claims weight domains equally.

Random deletion keeps fewer than half of the valid requests. Figure 8B asks how many valid requests would survive if each Local scope list were cut at random to full scope’s length. Suppose claim i has a local list of T_i requests, V_i of them valid, and full scope makes K_i requests. Drawing K_i of the local requests uniformly without replacement keeps $V_i K_i / T_i$ valid ones in expectation. The figure reports $\sum_i V_i K_i / T_i / \sum_i V_i$ within each split, and the two claims with empty lists contribute zero to both sums. The comparison uses only the saved list sizes and makes no model calls. A learned selection method could do better than random deletion.

Over all claims, precision rises and recall barely moves. Table 10 gives the rates over all 150 claims. The documents with valid chains are exactly the necessary ones, so justified-request precision equals one minus the unnecessary fraction. Every active obligation carries the same criticality weight, so critical-evidence recall equals obligation completeness, the share of active obligations whose evidence is complete. The score for inheriting source locations is zero for all systems, so the analysis does not show that the sources support every relation. If every claim on which either system lacks an output counts as a loss, justified-request precision still gains +0.112 on development, +0.103 on held-out claims and +0.107 overall, while critical-evidence recall changes by -0.261 , -0.289 and -0.278 . These recall changes come entirely from the claims counted as losses: wherever both systems have an output, recall is identical.

C.4 HOW EVERY PIPELINE COMPARES

CASEPATH holds documents back until a condition is resolved. Each method’s document-request scores are similar on development and held-out claims (Table 11). CASEPATH and its controls produce the most lists and hold many documents back, which the baselines rarely do (Table 12). In one recorded development trace, selected before any target was scored, arrears are assessed as true, so the missing cure notice is requested; whether the property is the family home is unknown, so a notice to the spouse stays on hold. The plan has seven documents to request now and four on hold, and its next action asks the question that decides whether another obligation applies.

Missing outputs decide the conservative contrasts. When every missing output counts against CASEPATH, including a baseline’s own failures, the advantage of Figure 13 disappears and every conservative contrast is negative (Table 13). Against every baseline, more than half of the held-out claims lack at least one of the two lists and score -1 whatever the other list contains.

Table 10: **Full process scope raises precision for nodes, relations and requests and leaves recall almost unchanged.** Full is CASEPATH; Local checks only each obligation’s own condition (Local scope). Every claim contributes, with the original failure penalties and weights; Compiled control matches Full exactly and is omitted.

	Development		Held out		All claims	
	Full	Local	Full	Local	Full	Local
<i>Process structure</i>						
Node precision	0.523	0.263	0.536	0.256	0.531	0.258
Node recall	0.542	0.542	0.544	0.526	0.543	0.532
Relation precision	0.676	0.378	0.679	0.368	0.678	0.372
Relation recall	0.374	0.374	0.381	0.370	0.379	0.372
<i>Requested evidence</i>						
Justified-request precision	0.704	0.331	0.732	0.318	0.721	0.323
Critical-evidence recall	0.635	0.635	0.645	0.627	0.641	0.630
Valid / all requests	259/275	259/592	384/385	376/853	643/660	635/1445
Claims with output	44	44	66	64	110	108

Table 11: **Checklist scores are similar on development and held-out claims for every method.** Document-request scores over all claims of each split; a failed output scores 0 for F_1 and 1 for the unnecessary fraction. Domains weigh equally, then the families and claims within them. The controls reuse CASEPATH’s assessments.

Method	Lists produced		Checklist $F_1 \uparrow$		Unnecessary fraction $\downarrow$	
	Dev.	Held out	Dev.	Held out	Dev.	Held out
CASEPATH	44	66	0.657	0.677	0.296	0.268
Direct	19	30	0.295	0.310	0.687	0.688
Document-first	17	24	0.247	0.249	0.732	0.737
Process-context	27	43	0.407	0.458	0.570	0.538
Rule-first	28	42	0.433	0.450	0.545	0.540
<i>Controls</i>						
Compiled control	44	66	0.657	0.677	0.296	0.268
Local scope	44	64	0.420	0.411	0.669	0.682

Table 12: **CASEPATH and its controls produce the most lists and hold back the most documents; Local scope’s lists carry more than twice as many requests.** All 150 claims. Lists is the number of outputs with raw counts; Correct/requested, Missed and Held (documents deferred until a condition is resolved) are summed over these outputs. Precision and recall cover every claim, with failure penalties and weights, so they differ from ratios of the raw counts.

Method	Lists	Correct/requested	Missed	Held	Precision $\uparrow$	Recall $\uparrow$
CASEPATH	110	643/660	95	2006	0.721	0.641
Direct	49	301/316	30	65	0.312	0.298
Document-first	41	237/247	39	139	0.265	0.236
Process-context	70	433/449	36	31	0.451	0.434
Rule-first	70	432/439	36	36	0.458	0.432
<i>Controls</i>						
Compiled control	110	643/660	95	2006	0.721	0.641
Local scope	108	635/1445	93	1387	0.323	0.630

Table 13: **Conventional contrasts favour CASEPATH; conservative contrasts, which count every missing output against it, do not.** Paired contrasts on the held-out claims; positive values favour CASEPATH, and the unnecessary fraction is sign-reversed so that a smaller fraction is positive.

CASEPATH against	Checklist F_1		Unnecessary fraction	
	Conventional	Conservative	Conventional	Conservative
Direct	+0.367	−0.758	+0.419	−0.744
Document-first	+0.429	−0.788	+0.469	−0.785
Process-context	+0.219	−0.673	+0.270	−0.657
Rule-first	+0.227	−0.671	+0.272	−0.650

C.5 PROVENANCE AND REPRODUCTION

The held-out claims are not a fresh hidden test. Product work had exposed their case inputs, and the prediction run ended without its complete audit of inputs and outputs, so we cannot prove that their reference labels were never accessed. The scoring step authenticates both complete sets of predictions before it reads any reference. The original scoring, the diagnosis of the condition names and document-request scoring each keep a separate record, and every record preserves the original failures. All reference containers have since been released unchanged under the recorded data licence, so the benchmark is now public.

Offline replay reproduces every Study B report exactly. The anonymous supplement contains all 1,050 prediction and failure records, the exact 150 reference containers, the original evaluator, the reporting code, the source commitments and the document-request scorer. Offline replay reproduces the predeclared report of each phase, the full predeclared report, every row of document-request scoring and the descriptive report, and their canonical JSON matches exactly. It rescores the saved outputs with no numerical tolerance, without regenerating them or calling the model. The supplement also holds a teaching example of the method, an invented repair policy whose 45 authored states `replay_teaching.py` re-executes and checks; it is labelled as teaching material and kept apart from the evidence.

D DATASHEETS FOR THE RELEASED DATASETS

Both datasets are in `data/` of the anonymous repository. Each has licence files, a README that serves as its datasheet and a script that checks every released file against recorded hashes. Data are under CC BY 4.0, except quoted third-party text (Study A’s policy, claim-form and model-condition passages and the form templates of Study B’s attachments), and code is under Apache 2.0.

D.1 STUDY A: 36 PAIRS OF THEFT CLAIMS

Each pair changes one fact and records the documents the change requires. A case is a short English story of a household theft followed by nine statements, one for each branch condition. In the first case of a pair all nine conditions are off; in the second, one of them is on, and nothing else changes. “No stolen item is separately scheduled in the policy with an insured value of CHF 1,000 or more”, for example, becomes “A stolen watch is separately scheduled in the policy with an insured value of CHF 4,000.” The reference lists the documents to add, grouped into requirements. Where the sources accept alternatives, it lists every acceptable combination, and asking for both alternatives counts as a wrong change (Appendix B.1). Every method receives the same 197 verbatim passages and the catalogue of 33 document types that the sources name (Tables 14 and 15). Code derived the references from a contract that records the decisions, conditions, facts, evidence and documents of the theft process that the sources support, with a verbatim quotation for every decision and condition. Two amendments, both made before the held-out pairs were read, turned “receipt and valuation” into “receipt or valuation” and added the bicycle purchase voucher. The release also holds the generator of the cases, the derivation of the references, the recorded checklists of the four methods and CASEPATH’s frozen representation; the representation is model output and not part of the reference.

The passages are real and cited; everything else is synthetic. The stories, dates and values are invented, and no person, policy or claim is real; the thresholds and periods in the condition statements come from the sources. Each passage is a short excerpt attributed to its document in the release. The policy terms, the claim form and the model conditions belong to their issuers and are excluded from the CC BY grant; Swiss federal enactments such as the VVG are not protected by copyright. In the frozen files, eight passages carry the label of a document that does not contain them, although each is verbatim in another document of the corpus. The release corrects these labels in its source notes and leaves the frozen files unchanged, and it also documents two stale convenience fields that the scorer does not read.

What the dataset does not cover. An agent built the contract behind the references from the sources, and no legal expert reviewed it. Every case switches on at most one condition, so interactions

Table 14: **The 36 pairs cover nine branch conditions, and each records the change its one fact requires.** Counts over all pairs.

<i>Corpus</i>		
Pairs	36: 9 development, 27 held out	
Cases	72 texts, 40 of them distinct (within a story, the nine cases with every condition off are identical)	
Branch conditions	9, in three groups: stolen property, events after the claim, settlement and recourse	
Stories	4 claim settings, the first for development	
Case text	English, 141 to 155 words	
<i>Reference</i>		
Required changes	44 (11 development, 33 held out), all additions	
Alternatives	one requirement (a receipt or an expert valuation), in 4 pairs	
Condition states	648, nine per case, fixed by construction	
<i>Shared inputs and tooling</i>		
Source passages	197 from 7 documents (Table 15)	
Document catalogue	33 types, all named in the sources	
Recorded outputs	4 methods, both checklists of all 36 pairs	
Scorer	frozen; recomputes every Study A result offline	
Licence	data CC BY 4.0 except the quoted policy terms, claim form and model conditions; code Apache 2.0	

Table 15: **The 197 passages are verbatim excerpts from seven public documents.** Three passages appear word for word in more than one document, so the counts sum to 201. The release gives the URL and file hash of each document.

Document	Edition	Passages
Insurance Contract Act (VVG), Arts. 38–41a, 45 and 46 (Swiss Confederation, 2024)	1 January 2024	26
CSS household contents policy terms (CSS Versicherung AG, 2024)	02.2024	67
CSS claim form, household contents and building (CSS Versicherung AG, 2026)	08.2026	19
AXA household policy terms (AXA Versicherungen AG, 2021)	10.2021	17
Generali household policy terms (Generali Allgemeine Versicherungen AG, 2026)	06.2026	25
Simpego household policy terms (Simpego Versicherungen AG, 2024)	12.2024	9
SVV model conditions for household contents (Schweizerischer Versicherungsverband SVV, 2022)	01.04.2022	38

between conditions are not tested, and every reference change is an addition, so a correct withdrawal cannot be measured. Only one requirement has alternative routes, and the held-out stories vary the claim setting, not the conditions. The held-out pairs were read once, after the method, data, analysis and criteria were frozen; they have been public since, so they remain a clean test only for work that keeps them out of development.

D.2 STUDY B: 150 TENANCY CLAIMS

Each claim pairs what an agent sees with a reference answer. An agent sees two files per claim. The intake packet holds the submission’s channel, date and language, the customer message as sent, and any attachments, embedded so that each claim file stands alone. The source registry lists the 136 to 140 units an agent may cite: the 101 entries of the three shared process templates, 19 passages of the Swiss Code of Obligations on tenancy (Swiss Confederation, 2026), 15 to 19 proposition-sized spans of the message and an inventory of the attachments. The reference is the generator’s own state for the case, and no system output was used to write it. It records the dispute type; the scenario flags that decide each branch; a process graph whose typed relations link decisions, facts, evidence capabilities, documents, steps and outcomes; the order of the steps; the evidence obligations, with the

expected state of each document; the process state at intake and the next action; and three or four source locators for each grounded concept (Table 16).

Table 16: The 150 claims cover three dispute types in two languages, and each carries a full structured reference. Counts over all claims; ranges are per claim.

<i>Corpus</i>	
Claims	150: 60 development, 90 held out
Dispute types	defects, lease terminations and rent increases, 50 claims each
Families	28 (11 development, 17 held out); no family crosses the split
Language	English 75, Swiss Standard German (de-CH) 75
Channel	email 70, portal 46, chat 24, phone note 10
Attachments	57 in 45 claims: 47 PDF notices, demands and forms, 10 photos
Customer message	138 to 209 words (median 179)
<i>Reference</i>	
Process graph	7,800 concepts (46 to 55) and 8,100 relations (47 to 58), each total including 600 that an output must not contain
Branch conditions	750 (3 to 7)
Obligation chains	1,397 (8 to 10), each from decision to fact, evidence and document
Order constraints	3,450 (20 to 25)
Document states	missing 961, conditional 395, provided but insufficient 41
Next action	process step 100, document request 35, escalation to an external specialist 15
<i>Tooling</i>	
Evaluator	33 metrics and 8 acceptance gates per claim
State-change track	48 variants: 12 base claims, each updated in four ways
Licence	data CC BY 4.0 except the cantonal form templates; code Apache 2.0

The released tools score structure, evidence and grounding. The evaluator scores 33 metrics over process structure, evidence, grounding and calibration, and accepts a claim’s output only if it passes eight gates, among them valid paths, complete obligations, correct document states and no forbidden elements. The reference output of every claim passes, and empty and request-everything outputs fail. The gates evaluate an output’s branch conditions against the reference scenario, so an output must state them in the scenario’s flags. The pipelines of Section 4.2 used the compiler’s own symbols, so the evaluator accepted none of them, and their requests were scored with the released document-request scorer instead (Appendices C.1 and C.2). The state-change track updates each of 12 base claims in four ways: it adds a sufficient document, makes a branch fact false, leaves that fact unresolved, or resolves every active fact. It scores nine endpoints, among them whether each request moves between now, later and never, and whether the unaffected part of the graph stays unchanged. The 24 development variants ship with their references. The references of the 24 held-out variants were regenerated with the released track code from the held-out claim references, and each matches the hash committed in the package.

What the dataset does not cover. The claims, messages, parties, images, process templates and references are synthetic. Two kinds of real material appear: the 19 legal passages, which Swiss copyright law does not protect, and the official Canton of Zurich forms on which 34 of the 47 PDF attachments are filled in, whose form text stays with its issuer and is excluded from the data licence. Every evaluated message named the intake mailbox of a real insurer as its recipient, in the raw message header that the pipelines did not read; the release replaces it with a fictional address of the same byte length, and a record binds each of the 300 changed files to the hash of its evaluated original, so that the original bytes can be checked once the name is disclosed after review. All claims are set in one canton. The reference captures each case at intake, and change over time appears only in the one-update variants. The next action, including escalation to a specialist, is labelled but not scored by the released evaluator, and it varies only among defect claims: every termination and rent-increase claim expects the step that preserves the challenge or extension deadline, and all 35 document requests and 15 escalations fall in defect claims. Each obligation has one acceptable document, so route choice is not tested. Images carry no labelled regions, and no field records amounts, liability or coverage. The held-out inputs were seen during product development (Appendix C.5), and their references were released after the evaluation of Section 4.2, so that split is not a hidden test; it remains a family-disjoint test split for anyone who keeps it out of training and tuning. The 60 development

references ship with the benchmark package, and the 90 held-out references are released beside it, each byte-identical to the file whose hash `cohort.json` committed before the evaluation.

E EXTENDED RELATED WORK

CASEPATH builds on keeping rules apart from the state of the case they apply to. LegalRuleML records norms and their sources (OASIS LegalRuleML Technical Committee, 2021), and Catala makes legislative computation executable (Merigoux et al., 2021). DMN formalises decisions, and CMMN represents case work that guards activate (Object Management Group, 2024; 2016). Guard-Stage-Milestone models describe decision- and knowledge-intensive processes around business artifacts (Vaculín et al., 2011), and DCR models of law check whether a business process complies with the law (López et al., 2020). Proactive enforcement deduces the actions that a policy of provisions and obligations requires and causes them in the target system (Basin et al., 2024), and commercial insurance software picks a claim’s document list from its service definition when the case is opened (ServiceNow, 2026). Those that run rules take the facts as structured data or, in proactive enforcement, as the events of the monitored system. LegalRuleML only encodes rules, and the DCR work checks process models, not cases. None reads the facts from a free-text case. The documented insurance step reads only the claim’s service definition, and it holds no request and asks no question. Guardrail agents bring rules to language-model agents: ShieldAgent compiles policy documents into probabilistic rule circuits and verifies an agent’s action trajectory against them (Chen et al., 2025), and GuardAgent turns safety requests into guardrail code that checks each proposed action (Xiang et al., 2025). Both judge actions that another agent chose. ShieldAgent also explains each violated rule and suggests countermeasures, but neither plans the actions, so neither would add a request that the other agent left out or ask about an open condition. CASEPATH has a model judge the conditions of a free-text case and derives the actions that the active process calls for, including questions for unresolved conditions, and we measure the request errors that follow.

Workflow research builds procedures or has agents follow them. PAGED is a benchmark for extracting procedural graphs from documents (Du et al., 2024). BREX is a benchmark for extracting business-rule flows from business documents, and its ExIde framework includes executable grounding through generated pseudo-code (Yang et al., 2026). Guideline2Graph converts clinical guidelines into executable decision graphs that keep their provenance (Kilic et al., 2026), and Graus (2026) tests whether structured intermediate representations help a model generate executable decision models from legal text. These systems build a process, rule set or decision model. CASEPATH also needs a process: agents built Study A’s rules from the sources, and code compiled Study B’s from templates. It then uses the process to decide what one case still needs. CANONIC decides whether content may enter a corpus the way a compiler decides whether a program is well formed (Hadley, 2026). It checks content against a grammar and has no case to plan for. FlowBench evaluates workflow-guided planning. It gives the agent a workflow as text, code or a flowchart and scores its tool calls and whether each session meets the user’s goals (Xiao et al., 2024). WorFBench evaluates whether a model can generate a workflow for a task (Qiao et al., 2025). Procedural Graphs place an agent at an active node, where a guidance model turns the surrounding graph into advice that biases the next action without dictating it, and a model refiner edits the graph by contrasting failed and successful trajectories (Lu et al., 2026). StateFlow separates process grounding, a state machine over the task, from the actions taken in each state (Wu et al., 2024), and ReAct interleaves reasoning and action without a process model (Yao et al., 2023). In τ -bench the domain policy is the agent’s system prompt, and a task succeeds when the final database state matches the goal and the agent’s replies contain the required information. In the original study the best agent, gpt-4o, succeeded on 61% of retail tasks and 35% of airline tasks (Yao et al., 2025). In FlowBench, τ -bench, Procedural Graphs and StateFlow the model chooses the actions, apart from a few fixed steps in StateFlow, and the workflow or policy informs that choice. In CASEPATH, once the process is fixed, the model only judges the case, and code decides every request. Procedural Graphs and StateFlow track one current node or state. CASEPATH evaluates every obligation at once, and under full process scope (Study B) it also checks all the branches that enclose it. We ask whether a changed fact changes exactly the evidence that its branch justifies.

The closest precedent is the evidence ledger of Kim (2026), in which branch conditions activate knowledge obligations, a model judges semantic warrant and code manages closure. Its obligations are what to retrieve to answer one query, and a model plans them for each query. Ours come from the

process, which is the same for every case. Its condition nodes also keep a branch’s obligations inactive until the condition is resolved, and a condition-evaluation tool call must resolve each one before the answer is final. Our conditions are judged from the case and may stay open in the plan: the request is held, and the condition becomes a question for the claimant. The paper describes no conditions that an obligation inherits from enclosing branches, and in Study B such conditions remove more than half of the requests. The ledger retrieves from a knowledge base, asks no person for documents and keeps no alternative routes. Its Warrant Judge checks whether a retrieved passage answers the query, a judgment close to ours on whether a document on file is adequate. The paper analyses deployment traces from one university knowledge base and releases no benchmark or code. Yu et al. (2026) compile the machine-readable constraints of standard operating procedures into pseudo-code whose rule routines return whether a constraint holds and the evidence for it, and a runtime shows the model only the active frame. The model still gives every verdict and chooses every tool call, and the authors call this enforcement “deliberately soft”. Once the process is built, the CASEPATH model only judges the conditions and, in Study B, the documents on file. Code combines these judgments, so every request traces to an obligation that the process makes active. Document planning adds its own problems: an obligation can apply before its evidence may be requested, a document on file can be inadequate, and several documents may have to be assessed together. The representation keeps alternative routes, and Study A shows what requesting all of them costs (Table 8). Study B tests the state of documents on file, but neither study tests timing, route choice or joint assessment: Study A requests every route and has no documents on file, and in Study B every timing condition is true and every capability has one route with one document (Appendix A).

Why no process-aware system is a baseline. No system cited here could run unchanged on our task (Table 17). Our cases need a list of documents to request, and we score each request. The workflow benchmarks and agents return tool calls, environment actions or a final database state, and none of the cited systems has public code that takes a process and a case and returns such a list. Some need inputs our cases lack. StateFlow’s authors note that it “requires humans to have a good understanding of a given task and build the model and prompts” (Wu et al., 2024). Compile-then-Page takes machine-readable constraints and was not tested on procedures written in natural language. Procedural Graphs start at a Start node, then find the active node by matching the agent’s last tool call, and fall back to the whole graph only when no node matches. Our task is one step with no tool calls, so their guidance would come from the Start node’s neighbourhood; the whole graph in the prompt, without the guidance model, is what Graph context tests. Evolving the graph needs scored successful and failed trajectories and a validation split. Guardrail agents need another agent’s actions to check. Compile-then-Page (“Artifacts will be released”) and the evidence ledger had no public code when we checked, and the Procedural Graphs paper links no official code.

We test what these systems share on the model side, with the same model, sources and cases as CASEPATH. Giving the model the rules and letting it decide is Direct, which in Study A reads the sources in full as a τ -bench agent reads its policy. Graph context adds CASEPATH’s graph and rules to the prompt, as FlowBench adds a workflow. In Study B every baseline reads the same compiled process, as the Compile-then-Page model reads its compiled procedure, and Process-context is told to resolve the process scope and obligations first and then choose the requests itself. Rule-first has the model derive condition-and-action rules and execute them itself. Listing evidence needs before planning, as the ledger’s planner does but without its code-kept states, is Evidence-first in Study A, which composes extraction, retrieval and planning, and Document-first in Study B. Compiled control recomputes CASEPATH’s decisions from the same judgments as one flattened formula, as a rule engine would, and matches them exactly (Section 4.2). We did not rebuild StateFlow’s state transitions, the self-evolving graph of Procedural Graphs, the paging runtime of Compile-then-Page, the evidence ledger or a guardrail filter over a baseline’s list, so our results do not show that CASEPATH beats these systems. They show that in Study A a model with CASEPATH’s graph in its prompt made 41 wrong checklist changes to CASEPATH’s 15, while missing fewer (Table 2), and that in Study B, at the same judgments, full process scope removed requests that do not apply and kept every valid one (Figure 7). Under the predeclared Study B rule, CASEPATH beats no baseline (Table 13).

Neuro-symbolic systems split the work in the same way. They let a language model translate a problem into a formal representation and leave the decision to an exact engine: Logic-LM hands a symbolic formulation to a solver (Pan et al., 2023), and Guan et al. (2023) have a model build a PDDL domain model that a sound planner then uses. CASEPATH applies this split to choosing actions in a workflow. Its process is built once for all cases and serves as the exact engine, and the model

assesses each case’s conditions. Our Rule-first baseline is the variant in which the model writes the rules and follows them itself; in Study A, agents write the rules and code runs them.

Grounding work supports answers, and information seeking decides what to look up. GraphCompliance aligns policy and context graphs for regulatory assessment (Chung et al., 2025), and CLER is a benchmark for reasoning about evidence in Chinese litigation (Bi et al., 2026). InfoGatherer uses evidential uncertainty to choose what to retrieve and what to ask (Taranukhin et al., 2026), Self-RAG learns when to retrieve (Asai et al., 2024), MediQ has a model hold back a diagnosis and ask follow-up questions when it is not confident (Li et al., 2024), and Ask-before-Plan predicts when a planning agent needs clarification (Zhang et al., 2024). GopherCite answers with quoted evidence (Menick et al., 2022), and chain-of-verification checks a draft answer with verification questions (Dhuliawala et al., 2024). API-Bank and BFCL evaluate function calling in tool use (Li et al., 2023; Patil et al., 2025). None of these systems has a process that says which requests a case justifies, and those that seek information retrieve or ask in order to answer a question or make a plan. CASEPATH asks when a condition that decides whether an obligation applies is still unresolved, requests documents when an active obligation’s fact is still open, quotes the case for each condition it judges, and keeps a chain from source to document for every request. We measure whether each evidence request is justified in the particular case.

Shortcut and validity research shaped our controls and outcomes. Work on partial-input baselines, annotation artefacts and shortcut learning motivates our input-free and wrong-condition controls (Kaushik & Lipton, 2018; Poliak et al., 2018; Gururangan et al., 2018; McCoy et al., 2019; Sugawara et al., 2018; Geirhos et al., 2020; Bowman & Dahl, 2021; Yuan et al., 2024). Code derived Study A’s references from a hash-bound contract that an agent built from the sources, and code scored the outputs without a model judge. This avoids judge biases such as preference leakage, where a judge model favours models trained on data from a related model (Li et al., 2026). Research on evaluation validity (Thomas et al., 2026), the finding that few papers on legal judgment prediction actually predict court decisions (Medvedeva & McBride, 2023), and the critique that legal benchmarks start from inputs that legal experts have already prepared (Lou & Shin, 2026) motivate our choice of document changes and requested documents as outcomes.

Table 17: **No cited process-aware system runs unchanged on our cases.** What each returns, what it would need on our cases, and why we did not run it. The last column names our closest test; (A) and (B) mark Study A and Study B.

System	Returns	Would need on our cases	Why not run; closest test
FlowBench (Xiao et al., 2024)	Tool calls in a dialogue, scored on tool invocations and on session success	A workflow as text, code or flowchart, and tools	A benchmark. The workflow in the prompt is Graph context (A); every baseline in B has the process in its prompt.
τ -bench (Yao et al., 2025)	Tool calls and replies, scored on the final database state and on required information in the replies	The policy as system prompt, tools and a simulated user	A benchmark. The policy text in the prompt is Direct (A).
StateFlow (Wu et al., 2024)	SQL, shell or household actions	A state machine and prompts built by hand for each task	A state machine for our processes would be our own design. Process-context (B) has the model resolve the process before choosing.
Procedural Graphs (Lu et al., 2026)	The next action, with advice from the active node	A tool call to find the active node (a one-step task stays at the Start node); scored runs and a validation split to evolve the graph	One list per case, no tool calls; no official code. The graph in the prompt, without the guidance model, is Graph context (A).
Compile-then-Page (Yu et al., 2026)	Tool calls on SOPBench, scored on the call sequence and final database state	Machine-readable dependency constraints for each user goal	Code not released. Every baseline in B has the compiled process in its prompt; the paging runtime is not tested.
Evidence ledger (Kim, 2026)	An answer to a knowledge-base query	Obligations a model plans for each query, and a retrieval tool	No code or benchmark. Listing evidence needs first, without code-kept states, is Evidence-first (A) and Document-first (B).
ShieldAgent, GuardAgent (Chen et al., 2025; Xiang et al., 2025)	Allow or deny for another agent’s action; ShieldAgent also explains violated rules and suggests countermeasures	Another agent’s actions to check	Could only filter a baseline’s list; not tested.
PAGED, BREX, Guideline2Graph, Graus (Du et al., 2024; Yang et al., 2026; Kilic et al., 2026; Graus, 2026)	A procedural graph, rule flow or decision model	Source documents. Generation takes no case; Graus runs the generated models on preset test scenarios	They build a process; CASEPATH’s agents do this step in A. Rule-first (B) has the model derive and run its own rules.
DMN or Catala rule engine (Object Management Group, 2024; Merigoux et al., 2021)	Decisions on typed inputs	Typed facts, while our cases are free text	Run on the model’s judgments, it would compute what Compiled control (B) computes, which matches CASEPATH exactly.
ServiceNow document lists (ServiceNow, 2026)	A document list per claim service definition, set when a case is opened	The commercial platform	Closed software; the documented step reads only the claim’s service definition.